חלק 1 – בחירת מערך הנתונים

מערך הנתונים שבחרנו לנתח בפרויקט הוא: "AI Jobs Posting Trends from 2019-2023".

המערך מתאר מאגר מידע של מודעות דרושים בתחום הבינה המלאכותית. הוא כולל 50,000 רשומות של מודעות דרושים בתחום הבינה המלאכותית בין השנים 2019-2023. לכל מודעה יש מידע על תפקיד, שכר, מיקום החברה, מקום מגורי העובד, אחוז עבודה מרחוק, מיומנויות נדרשות, השכלה, שנות ניסיון ופרטים רלוונטיים נוספים.

ישנם 18 פיצ'רים שונים המוצגים במערך הנתונים:

1. Job id – קטגוריאלי (ייחודי)
2. Job title – קטגוריאלי
3. Salary USD – נומרי שלם
4. Salary currency – קטגוריאלי
5. Experience level – קטגוריאלי (אורדינאלי)
6. Employment type – קטגוריאלי
7. Company location – קטגוריאלי
8. Company size – קטגוריאלי (אורדינאלי)
9. Employee residence – קטגוריאלי
10. Remote ratio – נומרי שלם
11. Required skills – טקסט חופשי (רשימה)
12. Education required – קטגוריאלי (אורדינאלי)
13. Years experience – נומרי שלם
14. Industry – קטגוריאלי
15. Posting date – תאריך (טקסט)
16. Benefits score – נומרי ממשי
17. Company name – קטגוריאלי
18. Remote category – קטגוריאלי (אורדינאלי)

בחרנו במערך זה משום שהוא עוסק בתחום חדשני ומתפתח ומשלב מגוון רחב של משתנים. הוא מאפשר לזהות מגמות בזמן, לבחון קשרים בין שכר, ניסיון, מיקום גאוגרפי, מיומנויות נדרשות ורמות עבודה שונות. המערך מכיל מגוון רחב של פיצ'רים מסוגים שונים בעזרתם ניתן לבצע ניתוח מעמיק ולענות על שאלות מחקר מעניינות ורלוונטיות לשוק העבודה.

חלק 2 – חקירה ראשונית של הנתונים

לפני שנוכל לנתח את הנתונים ולהציג וויזואליזציות שונות, נצטרך לבצע עיבוד מקדים ולקודד משתנים קטגוריאליים אורדינליים שנרצה לעבוד איתם - למשל רמת השכלה וניסיון מקצועי .

בשלבים הבאים של הפרויקט נרצה לנתח שאלות שסובבות סביב המיומנויות הנדרשות של המשרות השונות. במערך הנתון, עבור כל משרה מופיעה רשימה של מיומנויות נדרשות (טקסט, מופרד בפסיקים). על מנת שנוכל לעבוד עם פיצ'ר זה, נצטרך לבצע לו עיבוד מקדים. תחילה, ניצור עבור כל משרה מערך שמכיל את המיומנויות הנדרשות שלה, כך שכל תא מכיל מיומנות בודדת. לאחר שיש לנו את כל המערכים לכל המשרות, נפצל את עמודת המיומנויות לעמודות נפרדות, כך שכל עמודה תכיל משרה בודדת, לפי המערכים שיש לנו. לאחר מכן, נבצע קידוד בינארי עבור סוגי המיומנויות כדי שנוכל למדוד מרחקים ולבצע הערכות בחלקים הבאים.

עיבוד נוסף שנרצה לעשות - קידוד עבור משתנים קטגוריאליים אורדינליים:

1. Education required - מיפוי ערכים לדרגה מספרית (בסולם ערכים של 1-4).
2. Experience level – מיפוי ערכים לדרגה מספרית (בסולם ערכים של 1-4).
3. Remote category – מיפוי ערכים לדרגה מספרית (בסולם ערכים של 1-3).

כדי לחקור את המערך שלנו לעומק החלטנו לבנות התפלגויות של משתנים שונים שיעזרו לנו בניתוח הנתונים בשלבים הבאים. בחרנו לבדוק את המשתנים שאנו חושבות שיש להם קורלציה עם משתנה הבדיקה שלנו – המיומנויות הנדרשות:

1. מיומנויות נדרשות: עבור כל מיומנות שמופיעה במערך, מה מספר המשרות שפרסמו אותה כחלק מהמיומנויות הנדרשות שלהן. מהתפלגות הנתונים בוויזואליזציה ניתן לראות כי הכישורים הוותיקים והבסיסיים שולטים בשוק – טכנולוגיות ותיקות ויציבות מופיעות בהיקף גדול מאוד, מה שמעיד שהן עדיין הכרחיות כמעט לכל תפקיד בתחום. מכיוון שהמערך מתייחס לשנים 2021–2023, הכישורים החדשים (5 אחרונים ברשימה) מופיעים בשכיחות נמוכה ביחס לאחרים עקב כניסתם המאוחרת לשוק, ולכן הם מהווים ערכים קטומים מימין שאינם משקפים את מלוא הפוטנציאל שלהם (ייתכן כי הם עדיין נחשבים ככישורי נישה, או שרק מתחילים להשתלב בדרישות עבודה). לפיכך נעדיף להתמקד ב-45 הכישורים הראשונים ברשימה לניתוח מייצג של התקופה.
2. תעשיות: עבור כל תעשייה שמופיעה במערך, מה מספר המשרות ששייכות לאותה תעשייה. הגרף מראה שהתפלגות המשרות בין התעשיות די אחידה, עם פערים קטנים בלבד, מה שמעיד על ביקוש רחב למומחי בינה מלאכותית בכל ענפי המשק. תחומים כמו בריאות, תקשורת וייצור מובילים במעט במספר המשרות.
3. שכר (USD) – מהי התפלגות השכר עבור המשרות שפורסמו במערך. במרכז הטווח, הביקוש לעובדים (או ההצעות) מתפלג בצורה כמעט אחידה, כלומר אין ריכוז חזק סביב שכר אחד מסוים אלא טווח רחב של משכורות מקובל בשוק. שכר נמוך מאוד או גבוה מאוד כמעט ולא מופיע בפרסומים, מה שעשוי להעיד על כך שמרבית התפקידים נמצאים בשוק בינוני-גבוה של שכר.

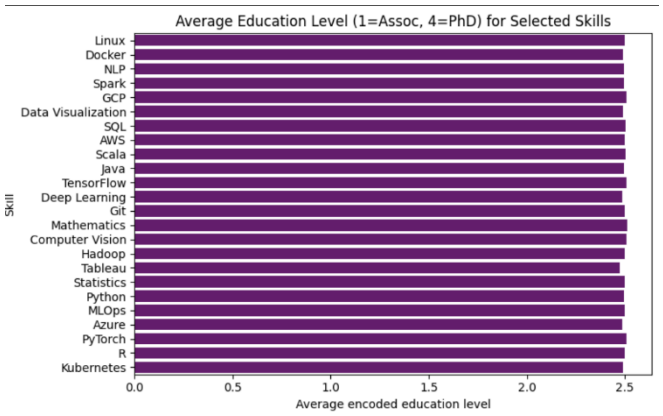
4. דרישות השכלה – עבור כל רמת ניסיון, מה מספר המשרות שדורשות רמה זו. רוב המשרות דורשות תואר שני או תואר ראשון, עם היקף כמעט זהה לשניהם. לעומת זאת, משרות הדורשות תואר שותף או דוקטורט נפוצות פחות, מה שמעיד על כך שהשכלה אקדמית מתקדמת אך לא בהכרח דוקטורט היא הדרישה הנפוצה ביותר בשוק.
5. ניסיון – עבור כל רמת ניסיון, מה מספר המשרות שדורשות רמה זו. ניתן לראות בגרף כי רוב המשרות מיועדות לעובדים ברמת כניסה ובדרג ביניים, כאשר מספר המשרות לשתי קבוצות אלו גבוה באופן משמעותי לעומת משרות בדרג בכיר ומנהלים. הדבר מעיד על ביקוש חזק לעובדים בתחילת הקריירה ובשלב הביניים, ופחות לתפקידים בכירים.

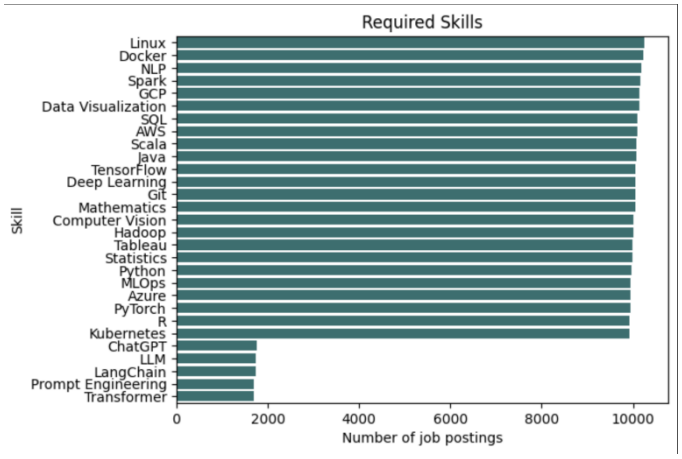
מכיוון שבחלקים הבאים נרצה להתמקד בקשרים בין משתנים, בדגש על קשרים בין מיומנויות נדרשות למאפיינים אחרים במערך הנתונים, נציג בנוסף את ההתפלגויות שיבחנו את הקורלציה בין המשתנים לצורך הניתוח המעמיק שנבצע בהמשך:

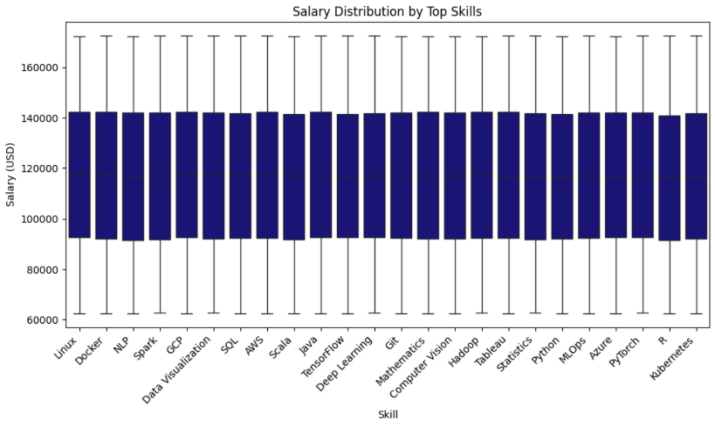
1. קשר בין מיומנויות נדרשות לתעשייה - הגרף מציג מפת חום של אחוז המשרות בכל תעשייה שבהן נדרש כל אחד מהכישורים המובילים. ניתן לראות שכישורים מסוימים מופיעים בשיעור גבוה יחסית כמעט בכל התעשיות, מה שמעיד על היותם כישורי ליבה רוחביים. לעומת זאת, כישורים אחרים בולטים יותר בתעשיות מסוימות, לדוגמה מתמטיקה בחינוך ואנרגיה. באופן כללי, הפיזור מראה שרוב הכישורים נדרשים במגוון רחב של תעשיות, אך יש התאמות ייחודיות בין חלק מהכישורים לתחומים ספציפיים.
2. קשר בין שכר למיומנויות נדרשות - הגרף מציג את התפלגות השכר לפי מיומנויות. רוב הכישורים מציגים טווח שכר דומה, עם חציון סביב כ־115 אלף דולר בשנה, וטווח כולל של כ־60 אלף עד 170 אלף דולר. אין הבדל משמעותי בשכר הממוצע בין הכישורים המובילים.
3. קשר בין ממוצעים מקודדים עבור המיומנויות הנדרשות בשלושה ממדים שונים:
   1- השכלה נדרשת – כמעט כל הכישורים דורשים באופן אחיד ממוצע בין תואר ראשון לתואר שני (רמה סביב 2.5 בסולם 1–4), מה שמעיד על דרישה להשכלה אקדמית גבוהה יחסית אך לא בהכרח לדוקטורט.
   2- רמת ניסיון נדרשת – רוב הכישורים נמצאים בטווח שבין דרג ביניים לדרג בכיר (כ־2.2 בסולם 1–4), כלומר נדרש ניסיון משמעותי אך לא ברוב המקרים ניסיון ניהולי בכיר.
   3- סוג משרה מבחינת עבודה מרחוק – רוב הכישורים נמצאים קרוב לערך 2.3 בסולם 1–3, מה שמרמז על שילוב בין עבודה היברידית לעבודה מרחוק מלאה, עם נטייה מסוימת לאפשרויות עבודה גמישות.

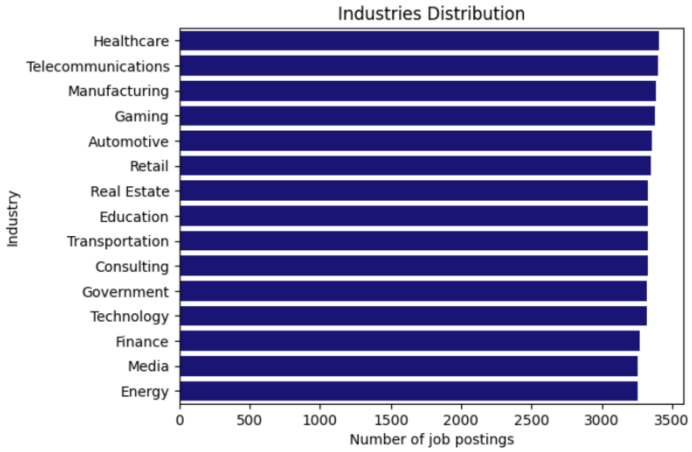
לסיכום, הכישורים המובילים דורשים לרוב השכלה אקדמית גבוהה, ניסיון מקצועי בינוני-גבוה, וגמישות מסוימת במיקום העבודה.

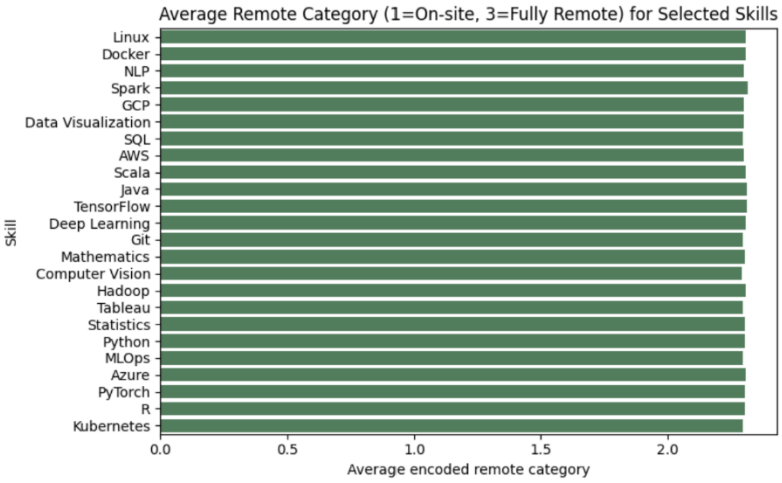
תמונות של הוויזואליזציות שבחרנו להציג:

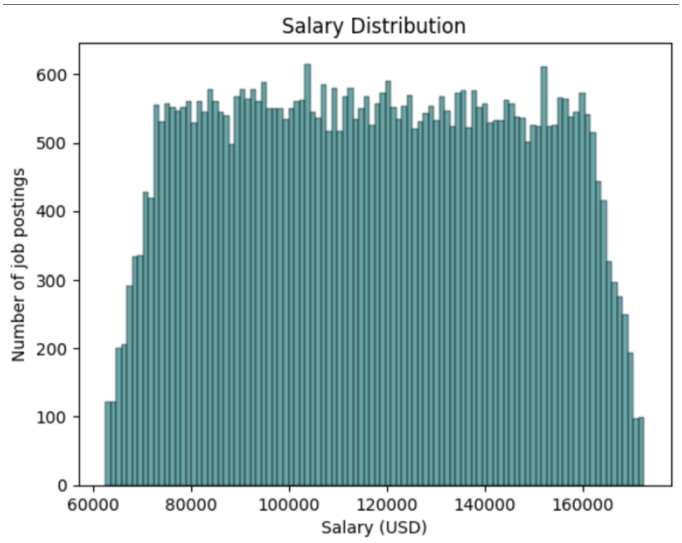


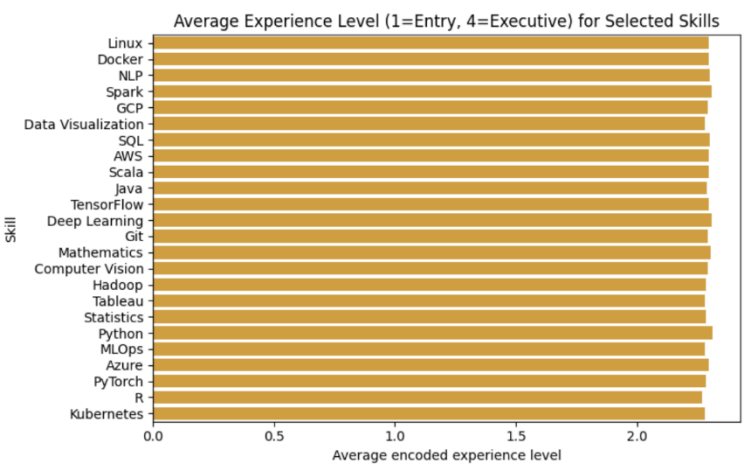


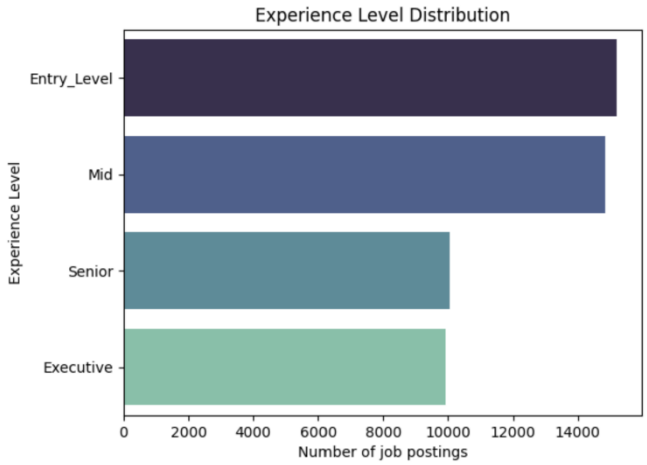


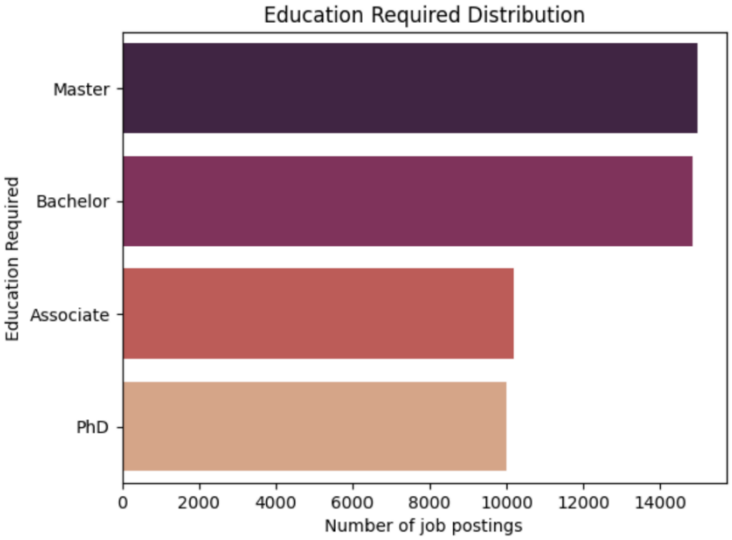


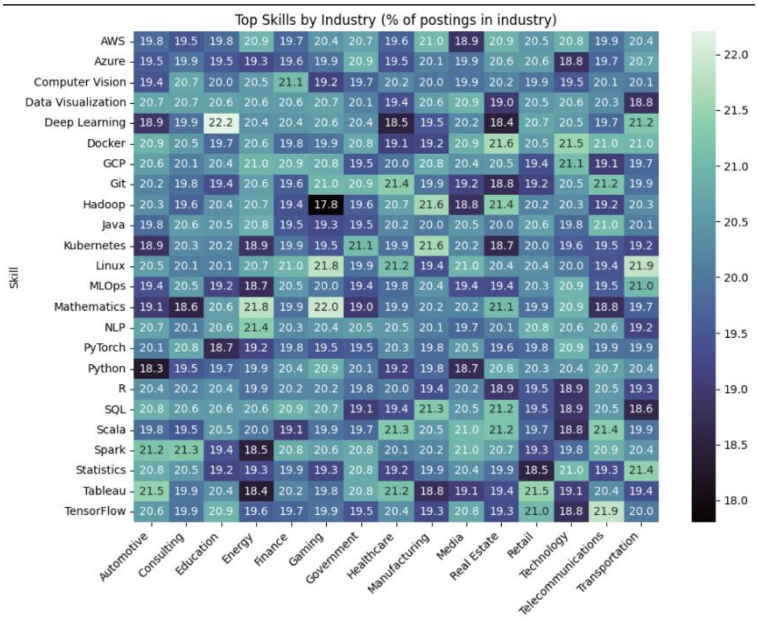





Average Experience Level (1=Entry, 4=Executive) for Selected Skills
Linux
Docker
NLP
Spark
GCP
Data Visualization
SQL
AWS
Scala
Java
TensorFlow
Deep Learning
Git
Mathematics
Computer Vision
Hadoop
Tableau
Statistics
Python
MLOps
Azure
PyTorch
R
Kubernetes
Skill
0.0
0.5
1.0
1.5
2.0
Average encoded experience level


Experience Level Distribution
Entry_Level
Mid
Senior
Executive
Experience Level
0
2000
4000
6000
8000
10000
12000
14000
Number of job postings


Education Required Distribution
Master
Bachelor
Associate
PhD
Education Required
0
2000
4000
6000
8000
10000
12000
14000
Number of job postings


Top Skills by Industry (% of postings in industry)
Skill
AWS 19.8 19.5 19.8 20.9 19.7 20.4 20.7 19.6 21.0 18.9 20.9 20.5 20.8 19.9 20.4
Azure 19.5 19.9 19.5 19.3 19.6 19.9 20.9 19.5 20.1 19.9 20.6 20.6 18.8 19.7 20.7
Computer Vision 19.4 20.7 20.0 20.5 21.1 19.2 19.7 20.2 20.0 19.9 20.2 19.9 19.5 20.1 20.1
Data Visualization 20.7 20.7 20.6 20.6 20.6 20.7 20.1 19.4 20.6 20.9 19.0 20.5 20.6 20.3 18.8
Deep Learning 18.9 19.9 22.2 20.4 20.4 20.6 20.4 18.5 19.5 20.2 18.4 20.7 20.5 19.7 21.2
Docker 20.9 20.5 19.7 20.6 19.8 19.9 20.8 19.1 19.2 20.9 21.6 20.5 21.5 21.0 21.0
GCP 20.6 20.1 20.4 21.0 20.9 20.8 19.5 20.0 20.8 20.4 20.5 19.4 21.1 19.1 19.7
Git 20.2 19.8 19.4 20.6 19.6 21.0 20.9 21.4 19.9 19.2 18.8 19.2 20.5 21.2 19.9
Hadoop 20.3 19.6 20.4 20.7 19.4 17.8 19.6 20.7 21.6 18.8 21.4 20.2 20.3 19.2 20.3
Java 19.8 20.6 20.5 20.8 19.5 19.3 19.5 20.2 20.0 20.5 20.0 20.6 19.8 21.0 20.1
Kubernetes 18.9 20.3 20.2 18.9 19.9 19.5 21.1 19.9 21.6 20.2 18.7 20.0 19.6 19.5 19.2
Linux 20.5 20.1 20.1 20.7 21.0 21.8 19.9 21.2 19.4 21.0 20.4 20.4 20.0 19.4 21.9
MLOps 19.4 20.5 19.2 18.7 20.5 20.0 19.4 19.8 20.4 19.4 19.4 20.3 20.9 19.5 21.0
Mathematics 19.1 18.6 20.6 21.8 19.9 22.0 19.0 19.9 20.2 20.2 21.1 19.9 20.9 18.8 19.7
NLP 20.7 20.1 20.6 21.4 20.3 20.4 20.5 20.5 20.1 19.7 20.1 20.8 20.6 20.6 19.2
PyTorch 20.1 20.8 18.7 19.2 19.8 19.5 19.5 20.3 19.8 20.5 19.6 19.8 20.9 19.9 19.9
Python 18.3 19.5 19.7 19.9 20.4 20.9 20.1 19.2 19.8 18.7 20.8 20.3 20.4 20.7 20.4
R 20.4 20.2 20.4 19.9 20.2 20.2 19.8 20.0 19.4 20.2 18.9 19.5 18.9 20.5 19.3
SQL 20.8 20.6 20.6 20.6 20.9 20.7 19.1 19.4 21.3 20.5 21.2 19.5 18.9 20.5 18.6
Scala 19.8 19.5 20.5 20.0 19.1 19.9 19.7 21.3 20.5 21.0 21.2 19.7 18.8 21.4 19.9
Spark 21.2 21.3 19.4 18.5 20.8 20.6 20.8 20.1 20.2 21.0 20.7 19.3 19.8 20.9 20.4
Statistics 20.8 20.5 19.2 19.3 19.9 19.3 20.8 19.2 19.9 20.4 19.9 18.5 21.0 19.3 21.4
Tableau 21.5 19.9 20.4 18.4 20.2 19.8 20.8 21.2 18.8 19.1 19.4 21.5 19.1 20.4 19.4
TensorFlow 20.6 19.9 20.9 19.6 19.7 19.9 19.5 20.4 19.3 20.8 19.3 21.0 18.8 21.9 20.0
Automotive
Consulting
Education
Energy
Finance
Gaming
Government
Healthcare
Manufacturing
Media
Real Estate
Retail
Technology
Telecommunications
Transportation
22.0
21.5
21.0
20.5
20.0
19.5
19.0
18.5
18.0

חלק 3 – בדיקת השערות

בחלק זה נצבע ניתוח מעמיק של הנתונים שלנו על ידי מבחן בדיקת השערות. ברצוננו לבדוק מה הקשר של מיומנויות נדרשות לכל מיני משתנים במערך. למשל, האם יש קשר בין מיומנות מסוימת לגובה שכר, לתעשייה מסוימת או לרמת ניסיון ספציפית.

השאלה שבחרנו לבחון היא: עבור המיומנות הנדרשת הנפוצה ביותר, האם ממוצע השכר עבור משרות שדורשות את מיומנות זו שונה ממוצע השכר עבור משרות שאינן דורשות את מיומנות זו.

בחרנו בשאלה זו בעקבות הרלוונטיות שלה לשוק העבודה בבינה מלאכותית - מעניין אותנו לבדוק האם באמת מיומנויות חמות יותר מתורגמות לשכר גבוה יותר. בנוסף, נרצה לבדוק האם קיים קשר סיבתי בין שני המשתנים, אחד קטגוריאלי (מיומנויות נדרשות – אשר קודדנו באופן בינארי לצורך הניתוח) ואחד נומרי (שכר). כלומר, האם דרישת מיומנות מסוימת גוררת שינוי בשכר המוצע. נוכל לבדוק שאלה זו בעזרת האמצעים שנלמדו בקורס.

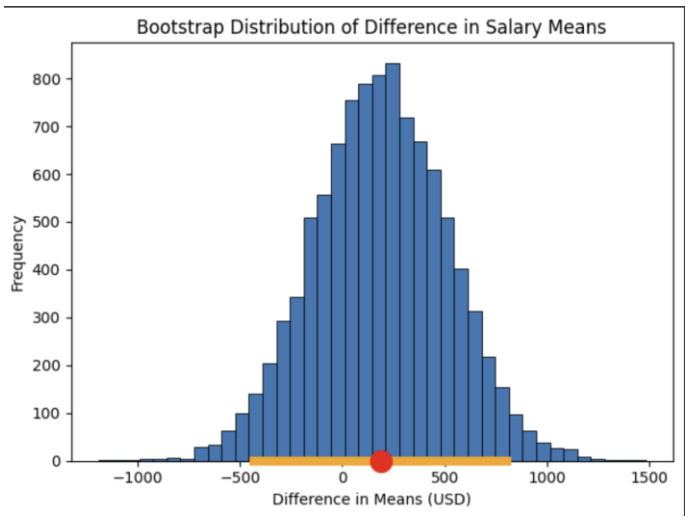
נבצע בדיקת השערות על השאלה שבחרנו לנתח באמצעות שיטת ה- bootstrap.

<u>השערת האפס:</u> הפרש הממוצעים בשכר של משרות שדורשות את המיומנות השכיחה ביותר למשרות שאינן דורשות את המיומנות השכיחה ביותר שווה לאפס.

<u>השערה אלטרנטיבית:</u> הפרש הממוצעים בשכר של משרות שדורשות את המיומנות השכיחה ביותר למשרות שאינן דורשות את המיומנות השכיחה ביותר שונה מאפס.

<u>תוצאות הבדיקה:</u> עבור הבדיקה שעשינו קיבלנו רווח סמך ברמת מובהקות של 95% - [822, -455] (הטווח יכול להשתנות במקצת עבור הרצות שונות – כחלק מעקרון הדגימה החוזרת בבוטסטראפ).

<u>מסקנה:</u> אפס נמצא בתוך הטווח של רווח הסמך שקיבלנו בתוצאות, לכן אנחנו לא דוחים את השערת האפס. לעומת זאת, אנחנו לא יודעות כיצד נאספו הנתונים במערך (באופן אקראי או לא), לכן לא ניתן להסיק בוודאות קשר סיבתי בין המיומנות הנפוצה ביותר לרמות השכר בקבוצות השונות. אולם ניתן להניח לפי התוצאות כי קיים ביניהם קשר נמוך, אם בכלל.



חלק 4 – סיווג / קלאסטרינג

בחלק זה בחרנו לבצע אשכול עבור הנתונים שלנו, על מנת לזהות מבני חלוקה בנתונים ולחלק את המשתנים הדומים לקבוצות הומוגניות, ללא תוויות. באלגוריתם שלנו נתמקד בחלוקת מיומנויות דומות, על בסיס רמת ההשכלה הנדרשת, רמת הניסיון, ומידת האפשרות לעבודה מרחוק במשרות הדורשות את המיומנות. נרצה לקבץ את הנתונים על מנת:

1. פילוח שוק: להבין אילו סוגי מיומנויות קשורים אחד לשני (בעלי מאפיינים דומים), ומה מאפיין כל אשכול מבחינת ניסיון / השכלה / אופי עבודה.
2. תכנון הכשרות: עבור כל אשכול להצביע על מיומנויות "שכנות" שחוזרות על עצמן, כדי לבנות מסלולי הכשרה והשלמה למועמדות.

יישום אפשרי של קלאסטרינג על נתונים אלו: שימוש באשכולות של המיומנויות כדי לבנות מספר "סלי כישורים" פרשניים. הסלים ישמשו גם להצעת נתיבי הכשרה ממוקדים וגם כפיצ'ר מסביר בניתוח השכר / הביקוש.

הפיצ'רים בהם בחרנו להשתמש בחלק זה:

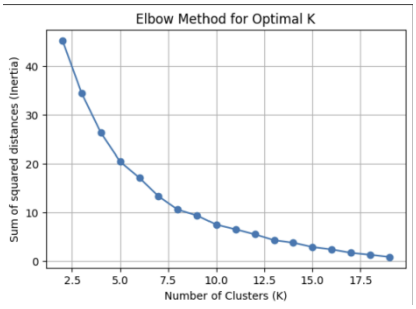
1. Education level – ממוצע על פני המשרות עם המיומנות הנבדקת. משקף את רמת ההשכלה הטיפוסית שנדרשת כשאותה מיומנות מופיעה.
2. Experience level – ממוצע על פני המשרות. משקף את הוותק הטיפוסי שמלווה את המיומנות הנבדקת.
3. Remote category – יחס המשרות שבהן העבודה היא מרחוק. נותן אינדיקציה עד כמה העבודה סביב המיומנות הנבדקת נוטה להיות גמישה / מרוחקת.

בחרנו בפיצ'רים אלו מכיוון שהם מתאימים לפרשנות. כלומר קל לתאר את הקשר הנבדק ביניהם. בנוסף, הם מוצגים כערכים מספריים במערך, ולכן מתאימים לאלגוריתם של k-means (ניתן לחשב את המרחק האוקלידי ביניהם). יתר על כן, שימוש בהם מצמצם את curse of dimensionality (קללת הממדים) לעומת שימוש בווקטורי one-hot גדולים כמו תעשייה / תפקידים.

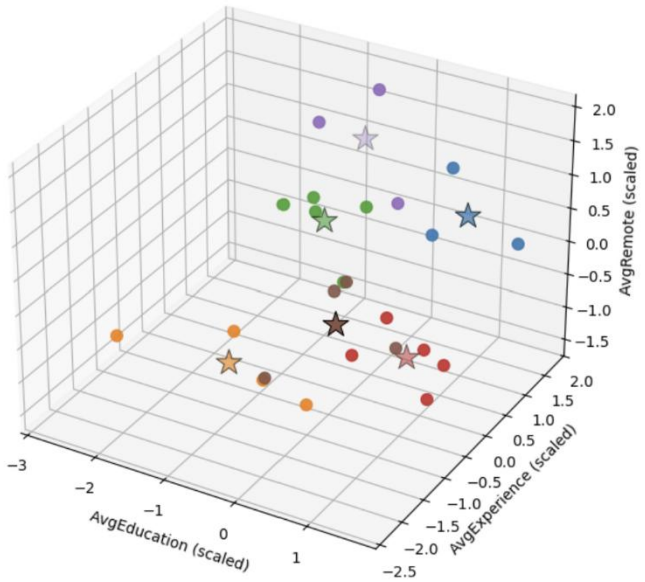
חילקנו את תהליך האשכול לשני חלקים. בחלק הראשון עשינו k-means עבור שלושת הפיצ'רים ביחד, ובחלק השני עשינו k-means עבור כל זוג של פיצ'רים (בכל פעם אחד מהפיצ'רים לא נכלל). ביצענו את שני החלקים על מנת לבדוק מה ההשפעה של הוספת ממד לתהליך, ואיך הוא משפיע על התוצאות.

<u>k-means עבור שלושת הפיצ'רים ביחד:</u>

עבור חלק זה בחרנו בערך k=6. על פי הגרף ניתן לראות כי אין מרפק ברור, לכן בחרנו משיקולי פרשנות בנקודה ממוצעת שלפניה השיפוע יחסית תלול, ואחריה השיפוע מתמתן.



בגרף שמציג את התוצאות (כל צבע מייצג אשכול, והכוכבים מייצגים את הצנטרואידים) ניתן לראות כי חלק מהאשכולות מרוכזים יחסית וחלק מפוזרים במרחב, ללא צפיפות וויזואלית ניכרת. למשל, האשכולות הירוק, האדום והכחול מרוכזים ברובם בסמוך למרכז האשכול. לעומת זאת, האשכולות הכתום, הסגול והחום משתרעים על שטח רחב יותר, ופחות מצומצמים כלפי המרכז.



מן התוצאות ניתן להניח כי קיים מדרג משותף של השכלה – ניסיון: באופן וויזואלי נקודות שנמצאות "גבוה" באחד הצירים avgEducation / avgExperience נוטות להיות גבוהות גם בציר השני. המדרג מרמז על זיקה חיובית בין רמת ההשכלה ורמת ניסיון כשהמיומנות הנבדקת מופיעה. נוסף על כך, יש מספר מיומנויות שנראות רחוקות מהצנטרואיד שלהן, כלומר ייתכן שמדובר בחריגות או רמיזה לתת מבנים.

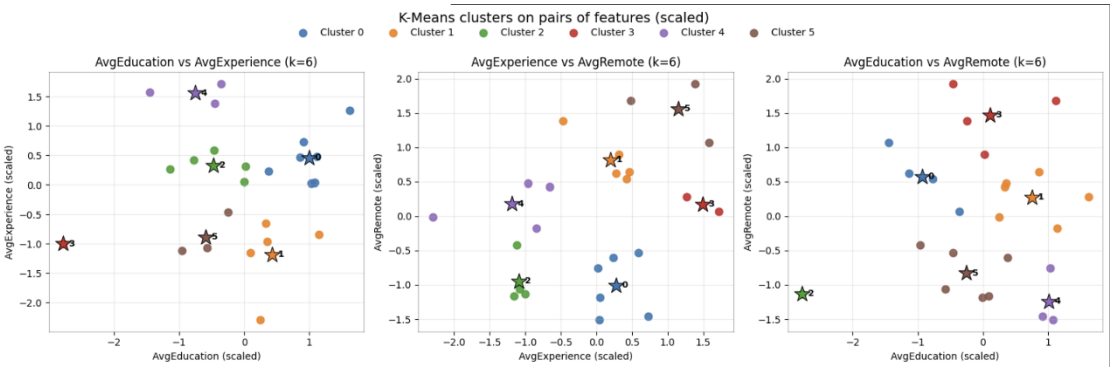
אם נרצה לחבר את התוצאות שקיבלנו ליישום שהצענו קודם לכן, אז על סמך התוצאות ניתן לבצע:

- תיאום ציפיות: מעבר מאשכול "נמוך – השכלה / ניסיון" לאשכול "גבוה" מצביע (ויזואלית) על צורך לא רק בכלים נוספים אלא גם בהעמקה תיאורטית / ניסיון פרקטי (קורסים / פרויקטים).
- התאמת סביבה: אם היעד הוא תפקידי remote / hybrid, יש עדיפות למיומנויות מאשכולות שמופיעות גבוה בציר ה remote. אם היעד הוא מעבדה / שטח, יש עדיפות לאשכולות "נמוכים" בציר הזה.

<u>k-means עבור זוגות של פיצ'רים:</u>

בחלק זה ביצענו בדיוק את אותם השלבים של החלק הראשון, רק שהפעם בשלוש פעמים, כך שכל פעם בודקים זוג אחר של פיצ'רים.

לפי תוצאות שלושת הגרפים ניתן לראות שעבור חלוקה בין 2 משתנים יש ויזואלית קשר יותר צפוף בין התצפיות לצנטרואידים (האשכולות יחסית ברורים יותר ומופרדים יותר).



בהשוואה לחלק הראשון (שלושת הפיצ'רים יחדיו) ניתן לראות כי עבור אשכולות שלא נראה בהם קשר גבוה בין התצפיות, נעדיף לנתח לפי זוגות, על מנת לגלות קשרים שאולי התפספסו בתלת ממד, כתוצאה מהשפעה של הפיצ'ר הנוסף. למשל, עבור האשכול החום, בגרף התלת ממדי (החלק הראשון) קשה לראות קשר ברור בין התצפיות באשכול. לעומת זאת, בחלק השני כאשר מחלקים לזוגות משתנים ניתן להבחין בקשר חיובי בין remote ל experience מה שקשה לראות בחלק הראשון. כלומר, השלכה זוגית מדגישה קשרים ספציפיים שנבלעו בתלת ממד (משמשת ככלי פרשנות משלים).

** עבור התצוגה התלת ממדית של הקלאסטרינג השתמשנו בבינה מלאכותית (לא נלמד בקורס כיצד להציג באופן הזה).

חלק 5 – סיכום ורפלקציה

בעיות אפשריות בנתונים:

- הטיית בחירה: אין מידע על גודל האוכלוסייה הכוללת והאם התצפיות שמוצגות נבחרו באופן אקראי ממנה. הטיה כזאת עלולה להשפיע על מסקנות הניתוח – המדגם הנבדק לא בהכרח מייצג את כלל האוכלוסייה הרלוונטית.
- מגבלת טווחי מדידה: הרבה משתנים במערך נמצאים בטווח ערכים מצומצם יחסית, מה שמקשה על מציאת קורלציה בין המשתנים.
- הטיית מדידה: אנחנו לא יודעות מאיזה אופן בוצע איסוף הנתונים. יכול לגרום לתוצאות לא נכונות.
- פרדוקס סימפסון: ייתכן כיוונים שונים של אפקטים בין שכבות של משתנה (למשל, סוגים שונים של תעשיות) במשקלים שונים יהפכו את התמונה הכוללת עבור המשתנה הכללי.
- K-means: לא קיבלנו מרפק ברור, ובחירת הערך שך k נעשתה משיקולי פרשנות, לכן ישנה אפשרות שהערך הנבחר אינו הערך המיטבי.

כיווני המשך – שאלות:

- "האם יש קשר בין remote category לשכר בתוך כל תעשייה / רמת ניסיון"
  השאלה עלתה בחלק הקלאסטרינג, כאשר בגרף האשכולות נראתה הפרדה חזותית לאורך ציר ה- remote.
- "האם בהינתן מיומנות נפוצה, יש הבדלי שכר בין תעשיות / שנים בהן המיומנות נדרשת לבין לא נדרשת".
  השאלה עלתה בבדיקת ההשערות באמצעות bootstrap, לאחר המסקנה שלא דוחים את השערת האפס.
- "האם בהינתן נתונים על מועמד, ניתן לקבוע אם הוא יתקבל למשרה מסוימת או לא".
  השאלה עלתה בחלק 4. רצינו לבצע ניבוי עבור השאלה, אך חסרים לנו נתונים על מועמדים למשרות. כלומר, לא ניתן לקבל תשובה על השאלה באמצעות המערך הנתון בלבד. לצורך מענה על השאלה, נצטרך לאסוף נתונים על מועמדים למשרות הנמצאות במערך הנתונים המקורי, כך שהמשתנים יהיו תואמים למשתני המשרות. עבור כל מועמד נצטרך מידע כדוגמת שנות ניסיון, רמת השכלה, לאיזו משרה מתמיין, מיומנויות עבודה, מיקום וכדומה.

final_project_212756233_342664018

August 15, 2025

[1]:
```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import numpy as np
import os
os.environ["OMP_NUM_THREADS"] = "1"
%env OMP_NUM_THREADS=1
```

```
env: OMP_NUM_THREADS=1
```

1 1

"AI Jobs Posting Trends from 2019-2023".

[2]:
```
# a block for understanding the data set structure
df = pd.read_csv('AI_Job_Posting_Trends.csv')
display(df.head())
display(df.info())
display(df.shape[0])
```

```
   job_id                job_title  salary_usd salary_currency  \
0  XYZ001      AI Software Engineer       76857             EUR
1  XYZ002             AI Architect       68860             GBP
2  XYZ003             AI Architect      120112             EUR
3  XYZ004  Principal Data Scientist      112812             EUR
4  XYZ005     AI Research Scientist       63245             USD

  experience_level employment_type company_location company_size  \
0        Executive        Contract            Japan            L
1      Entry_Level      Freelancer      Switzerland            M
2              Mid       Part Time    United States            S
3      Entry_Level       Full Time            India            L
4      Entry_Level       Part Time            Japan            S

  employee_residence  remote_ratio  \
0              China            50
1            Denmark            50
2              China            50
```

```
3              India            50
4             France            50

                                required_skills education_required  \
0                Git, SQL, Hadoop, GCP, Tableau                PhD
1          Git, Deep Learning, Java, SQL, MLOps             Master
2  Computer Vision, NLP, Linux, GCP, Statistics             Master
3              MLOps, Kubernetes, R, Linux, Git             Master
4          Tableau, Python, Scala, Docker, Java             Master

   years_experience        industry posting_date  benefits_score  \
0                 8  Transportation       3/9/19             6.6
1                 1          Energy      2/26/19             9.7
2                15     Real Estate       4/1/19             9.5
3                12          Energy       3/3/19             5.8
4                 2     Real Estate      6/23/19             7.3

    company_name remote_category
0  NeuralNet Inc          Hybrid
1  NeuralNet Inc          Hybrid
2   Visionary AI          Hybrid
3  NeuralNet Inc          Hybrid
4   Visionary AI          Hybrid
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 50000 entries, 0 to 49999
Data columns (total 18 columns):
 #   Column              Non-Null Count  Dtype
---  ------              --------------  -----
 0   job_id              50000 non-null  object
 1   job_title           50000 non-null  object
 2   salary_usd          50000 non-null  int64
 3   salary_currency     50000 non-null  object
 4   experience_level    50000 non-null  object
 5   employment_type     50000 non-null  object
 6   company_location    50000 non-null  object
 7   company_size        50000 non-null  object
 8   employee_residence  50000 non-null  object
 9   remote_ratio        50000 non-null  int64
 10  required_skills     50000 non-null  object
 11  education_required  50000 non-null  object
 12  years_experience    50000 non-null  int64
 13  industry            50000 non-null  object
 14  posting_date        50000 non-null  object
 15  benefits_score      50000 non-null  float64
 16  company_name        50000 non-null  object
 17  remote_category     50000 non-null  object
dtypes: float64(1), int64(3), object(14)
```

```
memory usage: 6.9+ MB
None
50000
```

’ :

1 job_id ( )

2 job_title

3 salary_usd

4 salary_currency

5 experience_level ( )

6 employment_type

7 company_location

8 company_size ( )

9 employee_residence

10 remote_ratio

11 required_skills ( )

12 education_required ( )

13 years_experience

14 industry

15 posting_date ( )

16 benefits_score

17 company_name

18 remote_category ( )

: 50,000

2 2

:

[3]:
```python
# Skills preprocessing:
# 1) Tokenize (comma-separated → list)
def to_list(s):
    parts = [p.strip() for p in s.split(",") if p.strip()]
    seen, out = set(), []
    for p in parts:
        if p not in seen:
            out.append(p); seen.add(p)
```

```
    return out
df["skills_list"] = df["required_skills"].map(to_list)

# 2) Get top skills
skills_long = df[["job_id", "skills_list"]].explode("skills_list").dropna()
skill_counts = skills_long["skills_list"].value_counts()
top_skills = skill_counts.index.tolist()

# 3) One-hot for the top-N skills
for skill in top_skills:
    df[skill] = df["skills_list"].apply(lambda lst: int(skill in lst))


# Encode experience_level (example order: Entry < Mid < Senior < Executive)
experience_map = {
    "Entry_Level": 1,
    "Mid": 2,
    "Senior": 3,
    "Executive": 4
}
df["experience_level_encoded"] = df["experience_level"].map(experience_map)

# Encode remote_category (example order: On Site < Hybrid < Fully Remote)
remote_map = {
    "On-site": 1,
    "Hybrid": 2,
    "Fully Remote": 3,
}
df["remote_category_encoded"] = df["remote_category"].map(remote_map)

# Encode education_required (example order: Associate < Bachelor < Master < PhD)
education_map = {
    "Associate": 1,
    "Bachelor": 2,
    "Master": 3,
    "PhD": 4
}
df["education_required_encoded"] = df["education_required"].map(education_map)
display(df.head())
```

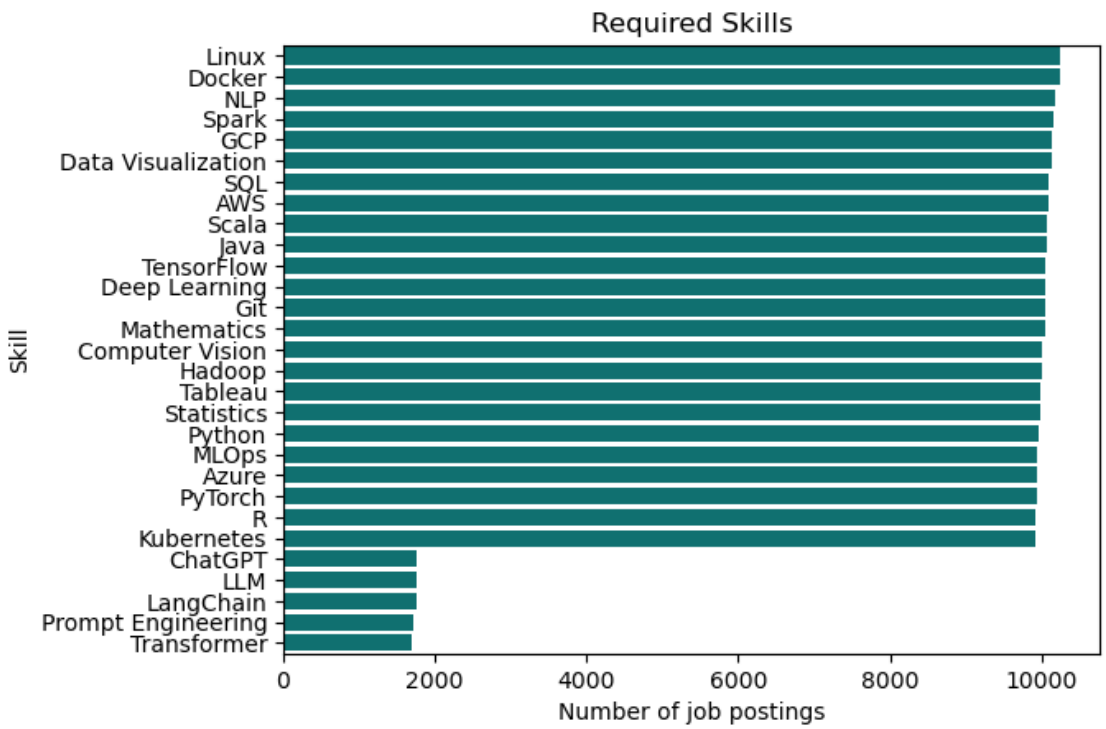
```
   job_id                job_title  salary_usd salary_currency  \
0  XYZ001      AI Software Engineer       76857             EUR
1  XYZ002             AI Architect       68860             GBP
2  XYZ003             AI Architect      120112             EUR
3  XYZ004  Principal Data Scientist      112812             EUR
4  XYZ005     AI Research Scientist       63245             USD

  experience_level employment_type company_location company_size  \
```

```
0        Executive        Contract            Japan            L
1      Entry_Level      Freelancer      Switzerland            M
2              Mid       Part Time    United States            S
3      Entry_Level       Full Time            India            L
4      Entry_Level       Part Time            Japan            S

  employee_residence  remote_ratio  …  R Kubernetes  ChatGPT LLM LangChain  \
0              China            50  …  0          0        0   0         0
1            Denmark            50  …  0          0        0   0         0
2              China            50  …  0          0        0   0         0
3              India            50  …  1          1        0   0         0
4             France            50  …  0          0        0   0         0

   Prompt Engineering Transformer experience_level_encoded  \
0                   0           0                        4
1                   0           0                        1
2                   0           0                        2
3                   0           0                        1
4                   0           0                        1

  remote_category_encoded  education_required_encoded
0                       2                           4
1                       2                           3
2                       2                           3
3                       2                           3
4                       2                           3

[5 rows x 51 columns]
```

:

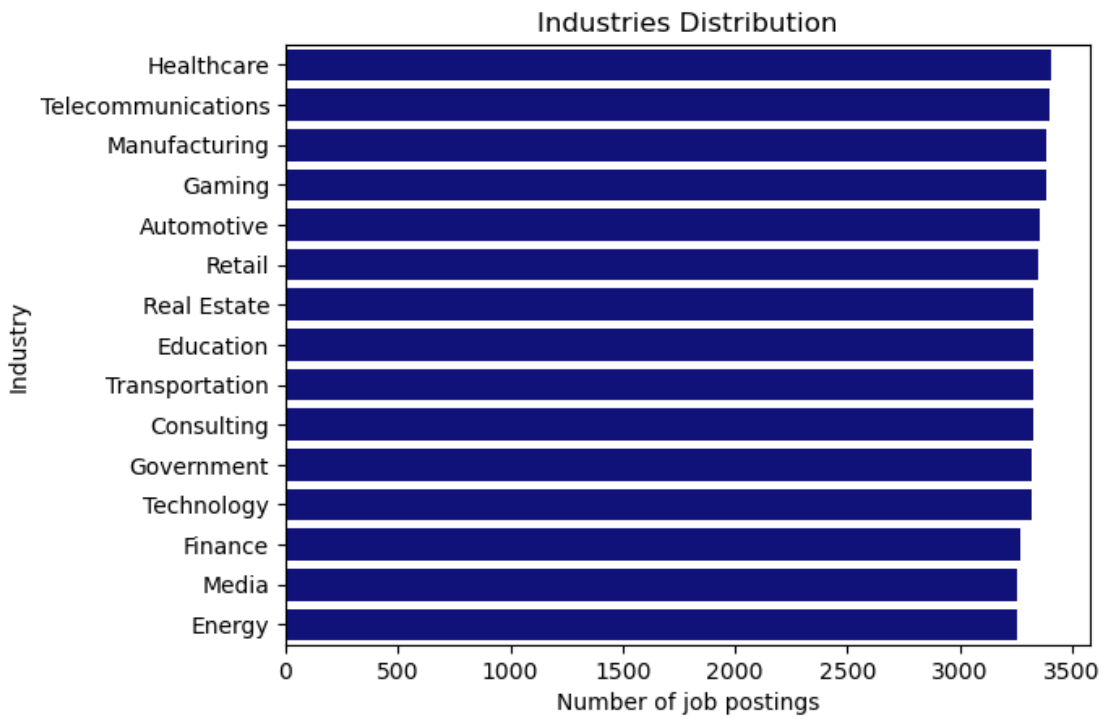
:

[4]:
```python
# 1) Required skills (bar chart)
skills = skill_counts.head(50).reset_index()
skills.columns = ["skill","count"]
sns.barplot(data=skills, x="count", y="skill", color="teal" )
plt.title("Required Skills")
plt.xlabel("Number of job postings")
plt.ylabel("Skill")
plt.show()
top24 = top_skills[:-5]
```



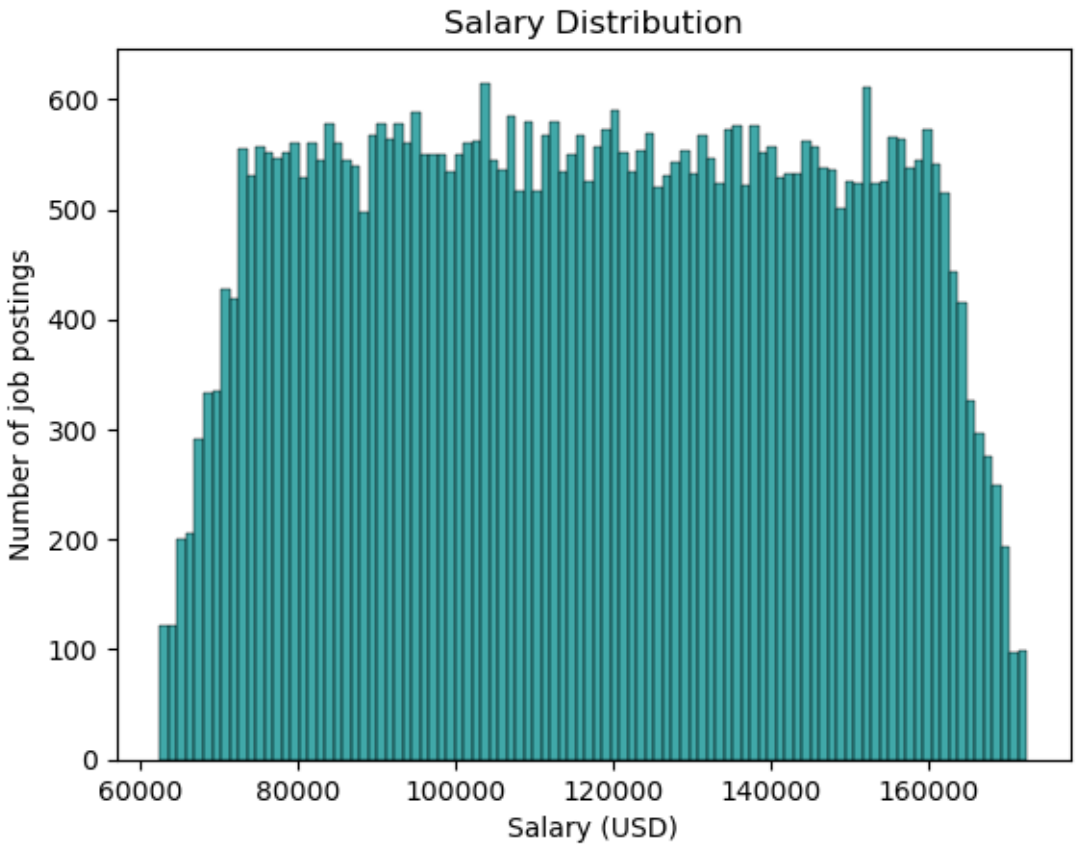
:

[5]:
```
# 2) Industries distribution (bar chart)
industries = df["industry"].value_counts().reset_index()
industries.columns = ["industry", "count"]
sns.barplot(
    data=industries,
    x="count",
    y="industry",
    color="darkblue" )
plt.title("Industries Distribution")
plt.xlabel("Number of job postings")
plt.ylabel("Industry")
plt.show()
```



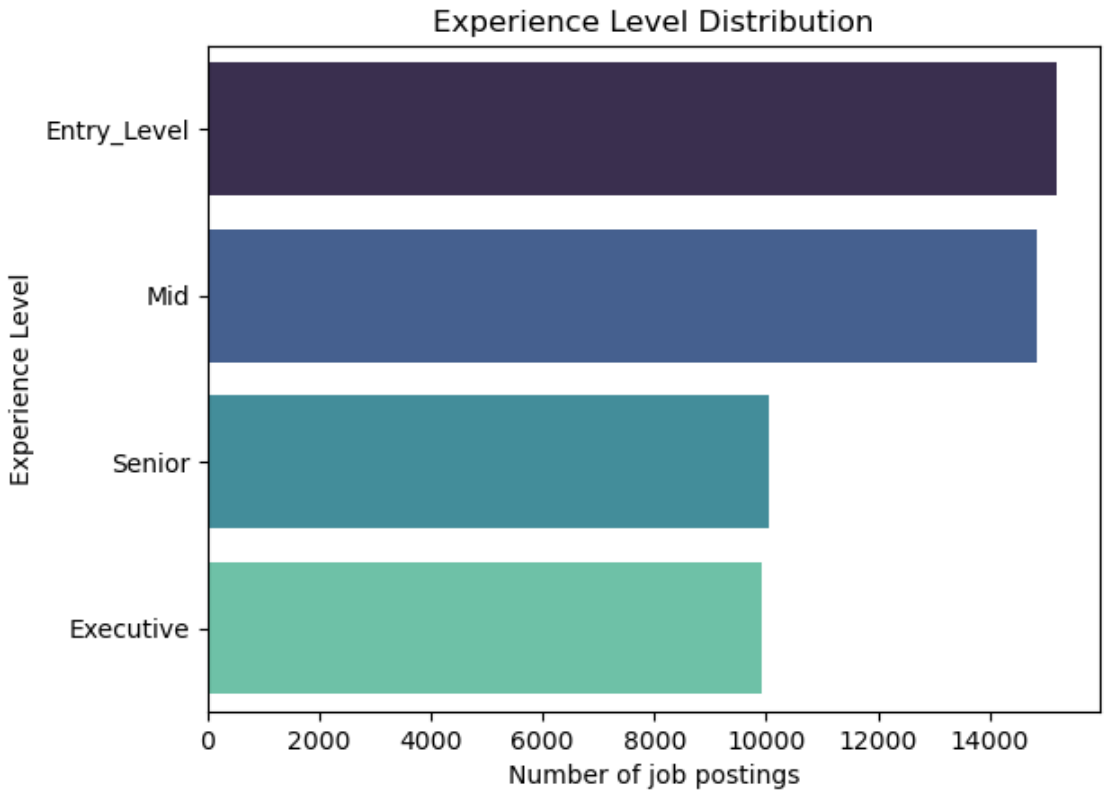
(USD):

[6]:
```
# 3) Salary distribution histogram
sns.histplot(
    data=df['salary_usd'],
    bins=100,
    color="darkcyan"
)
plt.title("Salary Distribution")
plt.xlabel("Salary (USD)")
plt.ylabel("Number of job postings")
plt.show()
```



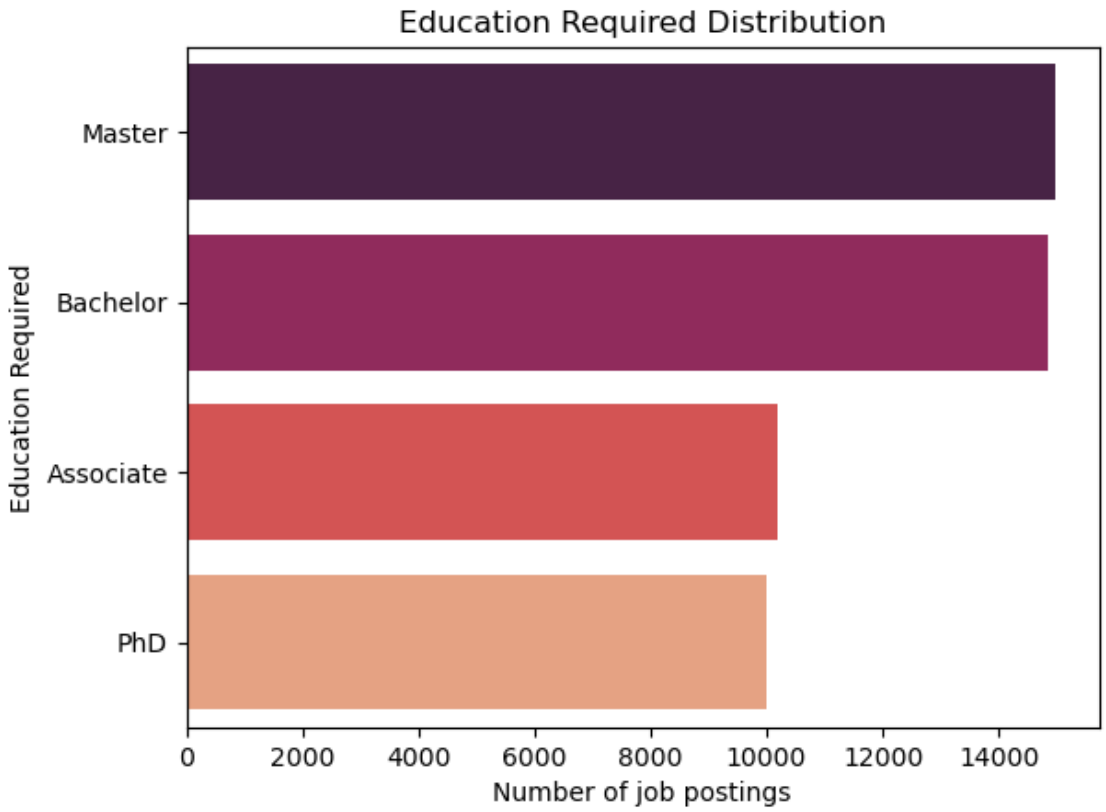
:

[7]:
```
# 4) Experience level distribution
exp_counts = df["experience_level"].value_counts().reset_index()
exp_counts.columns = ["experience_level", "count"]

sns.barplot(
    data=exp_counts,
    x="count",
    y="experience_level",
    hue="experience_level",
    palette="mako",
    legend=False
)
plt.title("Experience Level Distribution")
plt.xlabel("Number of job postings")
plt.ylabel("Experience Level")
plt.show()
```



:

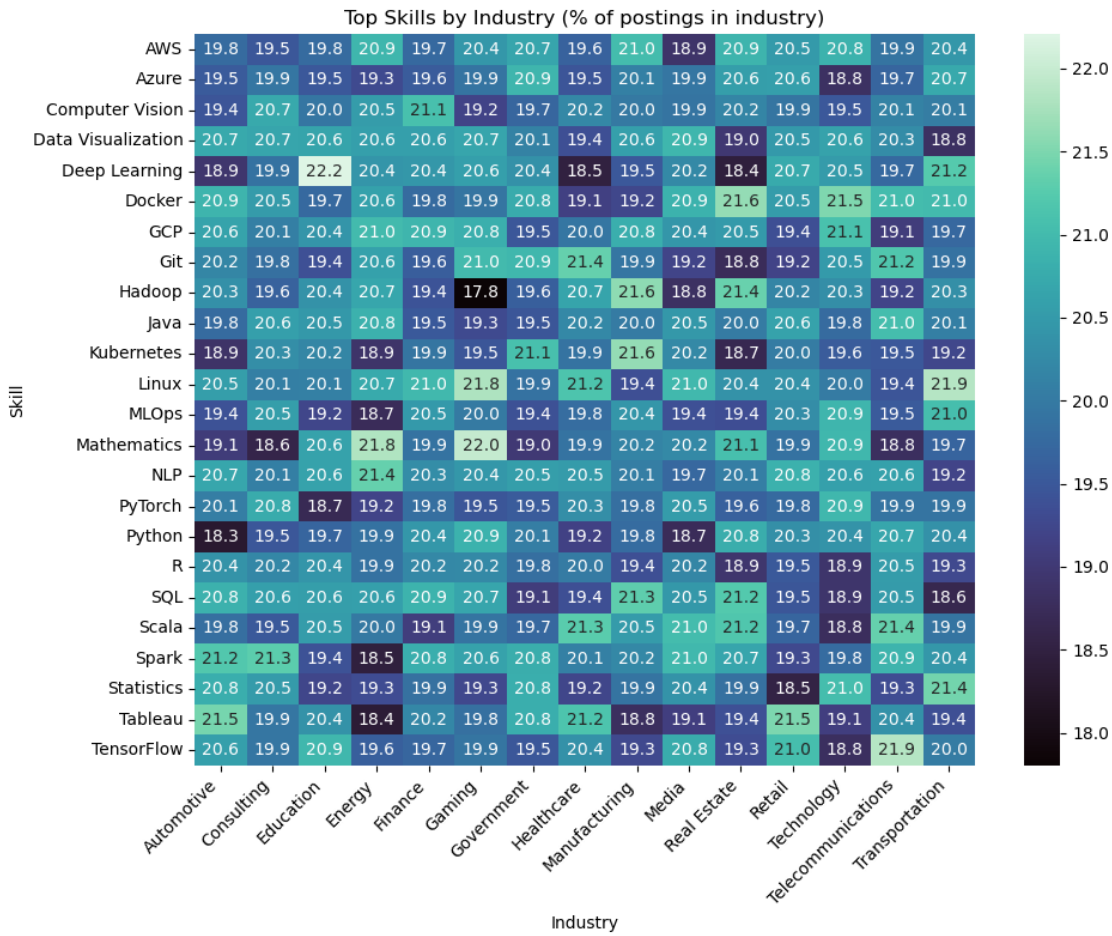
[8]:
```
# 5) Education required distribution
exp_counts = df["education_required"].value_counts().reset_index()
exp_counts.columns = ["education_required", "count"]

sns.barplot(
    data=exp_counts,
    x="count",
    y="education_required",
    hue="education_required",
    palette="rocket",
    legend=False
)
plt.title("Education Required Distribution")
plt.xlabel("Number of job postings")
plt.ylabel("Education Required")
plt.show()
```



:

[9]:
```
# 1) Skills vs. Industry (heatmap of percentages)
skills_for_heatmap = top24

# Melt to long format
skills_industry = df.melt(
    id_vars=["industry"],
    value_vars=skills_for_heatmap,
    var_name="skill",
    value_name="has_skill"
)

# Keep only rows where job requires the skill
skills_industry = skills_industry[skills_industry["has_skill"] == 1]

# Count postings per (skill, industry)
skill_ind_counts = skills_industry.groupby(["skill", "industry"]).size().
 ↪unstack(fill_value=0)

# Divide each industry column by its total postings (to get percentages)
```

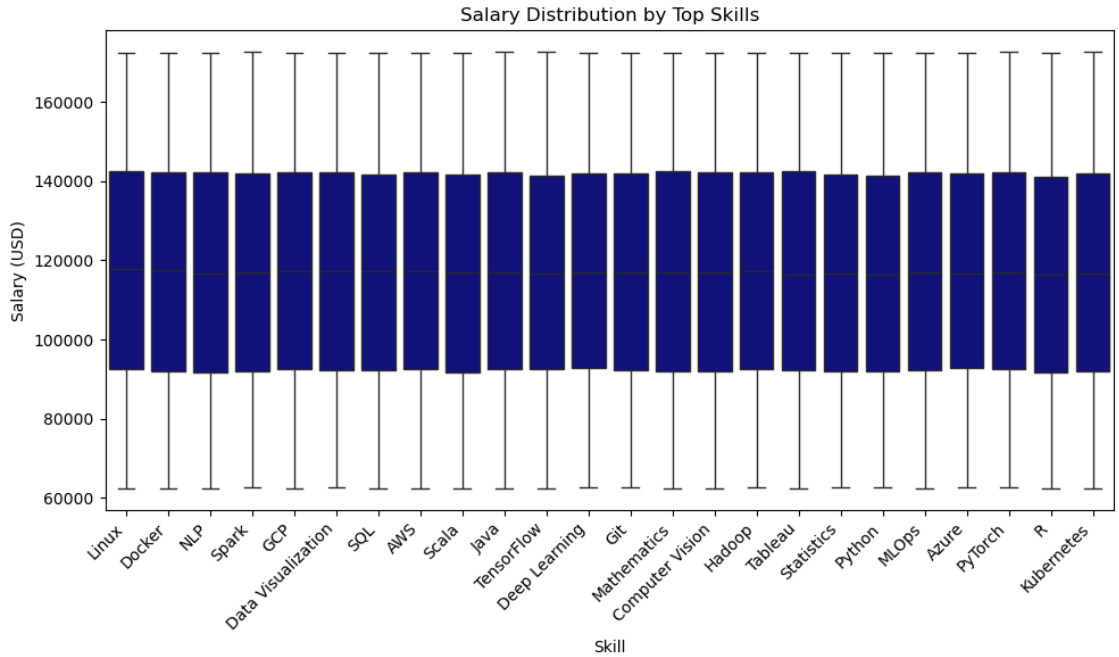
```
industry_totals = df["industry"].value_counts()
skill_ind_pct = skill_ind_counts.divide(industry_totals, axis=1) * 100  #␣
↪percentage

# Plot heatmap
plt.figure(figsize=(10, 8))
sns.heatmap(skill_ind_pct, cmap="mako", annot=True, fmt=".1f")
plt.title("Top Skills by Industry (% of postings in industry)")
plt.xlabel("Industry")
plt.ylabel("Skill")
plt.xticks(rotation=45, ha="right")
plt.tight_layout()
plt.show()
```



:

[10]:
```
# Salary distribution in different skills
salary_col = "salary_usd"
```

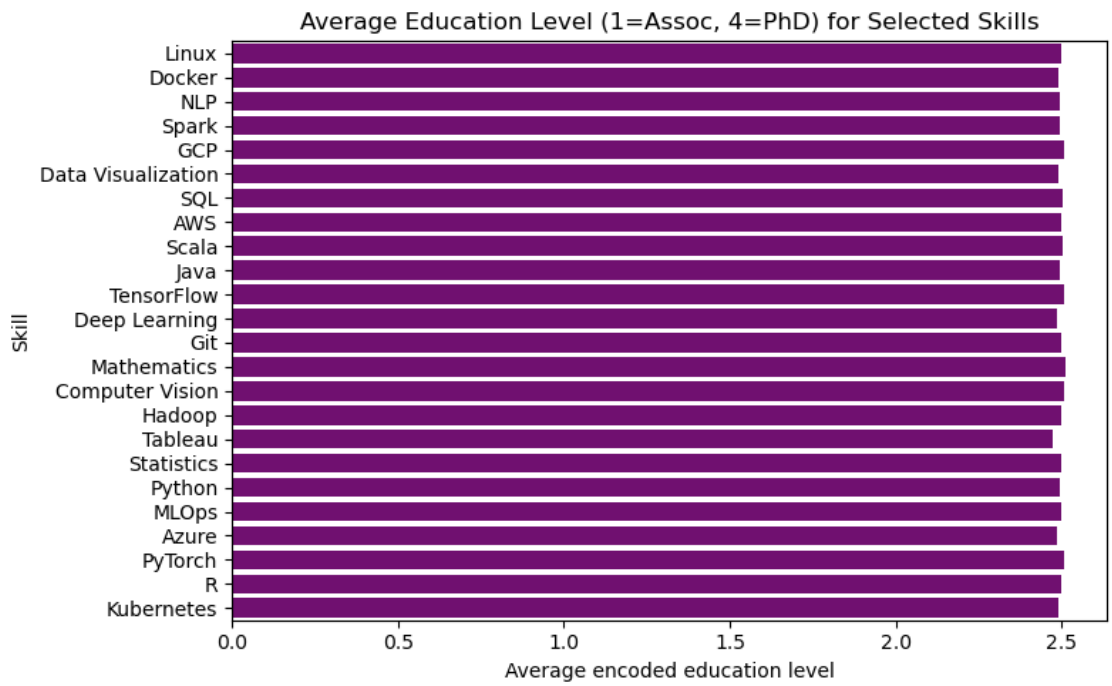
```
skills_for_salary = top24
skill_salary_data = []

for skill in skills_for_salary:
    # Jobs that require this skill
    skill_jobs = df[df["skills_list"].apply(lambda lst: isinstance(lst, list)␣
 ↪and (skill in lst))]
    # Count each job once
    skill_jobs_unique = skill_jobs.drop_duplicates(subset=["job_id"])
    # Keep salary as a DataFrame and tag with the skill name
    temp = skill_jobs_unique[[salary_col]].copy()
    temp["skill"] = skill
    skill_salary_data.append(temp)

# Combine and clean
skills_salary_df = pd.concat(skill_salary_data, ignore_index=True)
skills_salary_df = skills_salary_df.dropna(subset=[salary_col])


plt.figure(figsize=(10, 6))
sns.boxplot(
    data=skills_salary_df,
    x="skill",
    y=salary_col,
    color="darkblue"
)
plt.title("Salary Distribution by Top Skills")
plt.xlabel("Skill")
plt.ylabel("Salary (USD)")
plt.xticks(rotation=45, ha="right")
plt.tight_layout()
plt.show()
```



:

[11]:
```
# Education level distribution in different skills
skills_for_plot = top24

# Calculate average encoded education level for each skill
skill_avg_edu = {}
for skill in skills_for_plot:
    if skill in df.columns:
        # Take only jobs that require this skill
        mask = df[skill] == 1
        avg_val = df.loc[mask, "education_required_encoded"].mean()
        skill_avg_edu[skill] = avg_val

# Convert to DataFrame for plotting
plot_df = pd.DataFrame(list(skill_avg_edu.items()), columns=["Skill",␣
 ↪"AvgEducation"])


# Plot
plt.figure(figsize=(8, 5))
sns.barplot(
    data=plot_df,
    x="AvgEducation",
    y="Skill",
    color="purple"
```

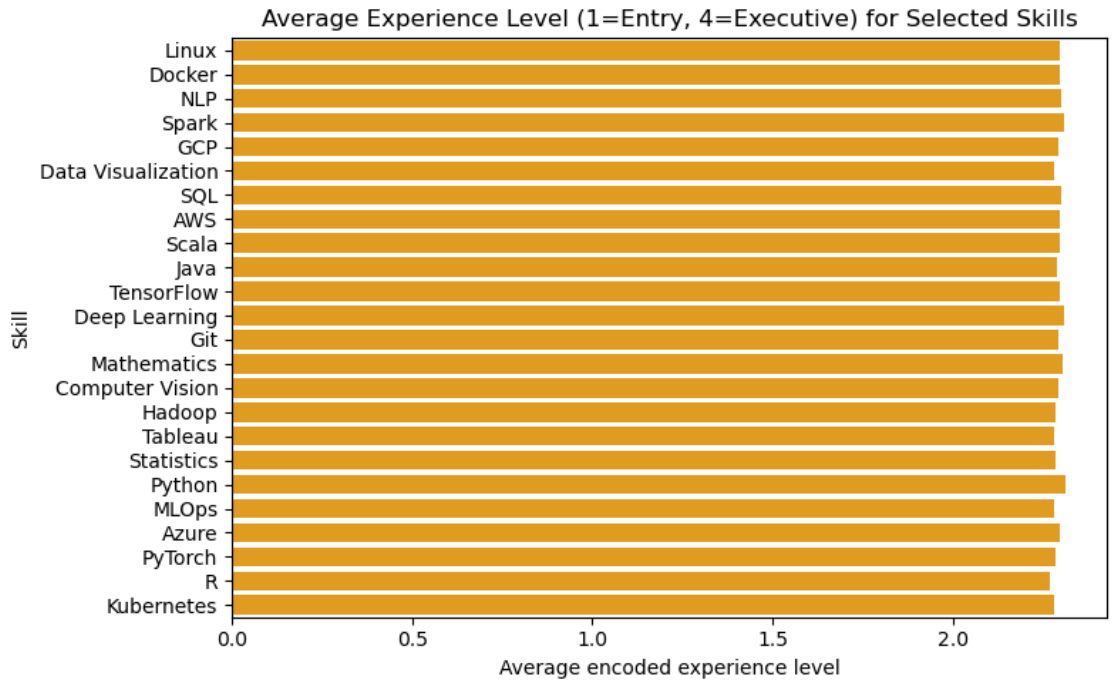
```
)
plt.title("Average Education Level (1=Assoc, 4=PhD) for Selected Skills")
plt.xlabel("Average encoded education level")
plt.ylabel("Skill")
plt.tight_layout()
plt.show()
```



: -1 -2 -3

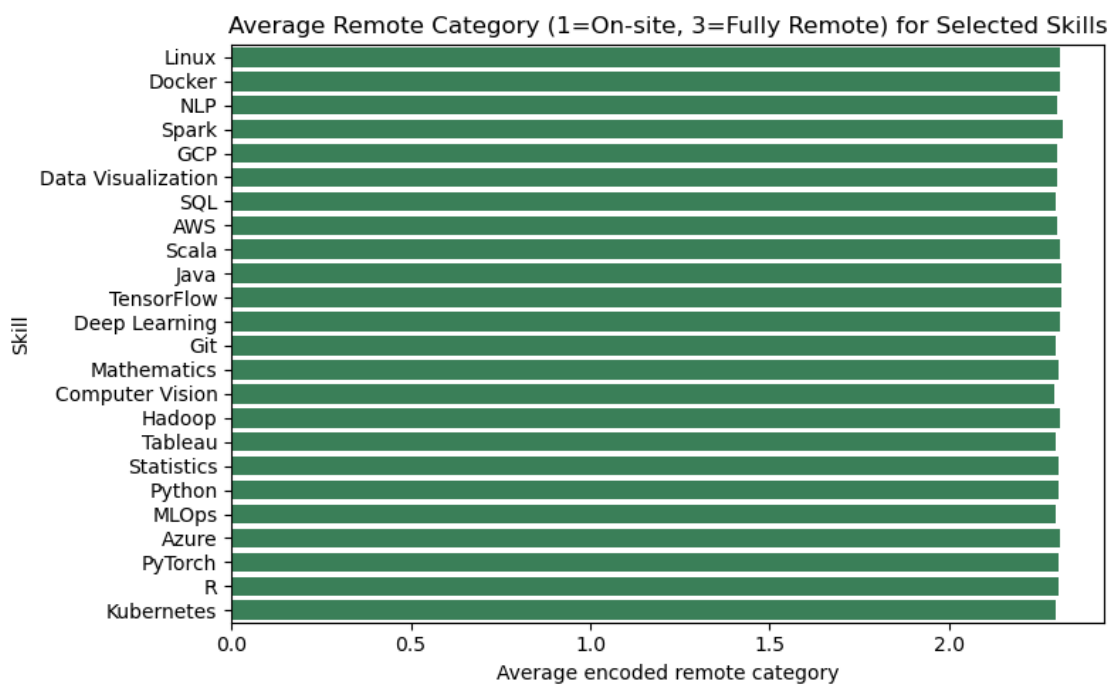
[12]:
```
# Experience level distribution in different skills
skills_for_plot = top24

# Calculate average encoded experience level for each skill
skill_avg_exp = {}
for skill in skills_for_plot:
    if skill in df.columns:
        # Take only jobs that require this skill
        mask = df[skill] == 1
        avg_val = df.loc[mask, "experience_level_encoded"].mean()
        skill_avg_exp[skill] = avg_val

# Convert to DataFrame for plotting
plot_df_exp = pd.DataFrame(list(skill_avg_exp.items()), columns=["Skill",␣
↪"AvgExperience"])
```

```
# Plot
plt.figure(figsize=(8, 5))
sns.barplot(
    data=plot_df_exp,
    x="AvgExperience",
    y="Skill",
    color="orange"
)
plt.title("Average Experience Level (1=Entry, 4=Executive) for Selected Skills")
plt.xlabel("Average encoded experience level")
plt.ylabel("Skill")
plt.tight_layout()
plt.show()
```



[13]:
```
# Job "remoteness" degree distribution by skills
skills_for_plot = top24

# Calculate average encoded remote category for each skill
skill_avg_remote = {}
for skill in skills_for_plot:
    if skill in df.columns:
        mask = df[skill] == 1
        avg_val = df.loc[mask, "remote_category_encoded"].mean()
        skill_avg_remote[skill] = avg_val
```

```
# Convert to DataFrame for plotting
plot_df_remote = pd.DataFrame(list(skill_avg_remote.items()), columns=["Skill",␣
↪"AvgRemote"])

# Plot
plt.figure(figsize=(8, 5))
sns.barplot(
    data=plot_df_remote,
    x="AvgRemote",
    y="Skill",
    color="seagreen"
)
plt.title("Average Remote Category (1=On-site, 3=Fully Remote) for Selected␣
↪Skills")
plt.xlabel("Average encoded remote category")
plt.ylabel("Skill")
plt.tight_layout()
plt.show()
```



3 3

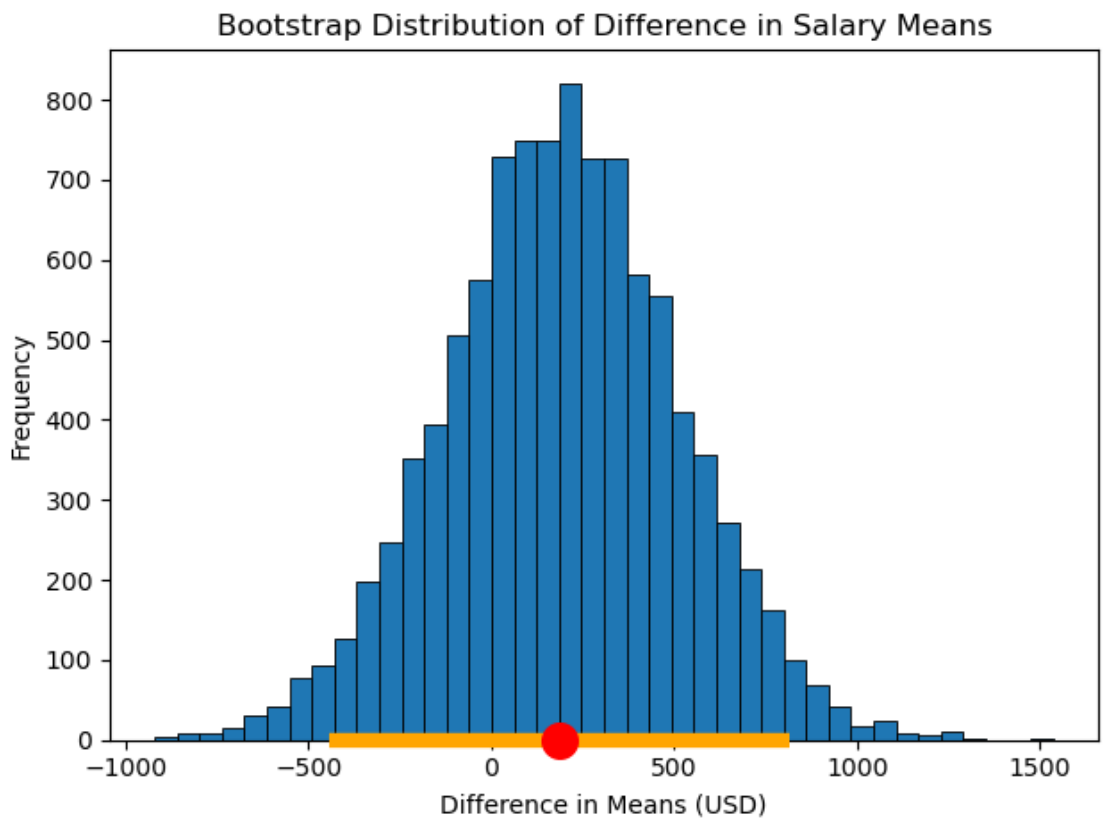
bootstrap.

[14]:
```
def bootstrap_mean(sample, mask, col_name, num_replications):
    sample_mask = mask.loc[sample.index]
    with_skill = sample.loc[sample_mask, col_name].dropna().
 ↪reset_index(drop=True)
    without_skill = sample.loc[~sample_mask, col_name].dropna().
 ↪reset_index(drop=True)
    sample_size_with = with_skill.shape[0]
    sample_size_without = without_skill.shape[0]
    bootstrap_means = np.empty(num_replications)
    for i in range(num_replications):
        bootstrap_sample_with = with_skill.sample(sample_size_with,␣
 ↪replace=True)
        bootstrap_sample_without = without_skill.sample(sample_size_without,␣
 ↪replace=True)
        bootstrap_means[i] = bootstrap_sample_with.mean() -␣
 ↪bootstrap_sample_without.mean()
    return bootstrap_means
```

[15]:
```
#             :
min_per_group = 50
top_skill = None
for s in top_skills:
    if s in df.columns:
        m = (df[s] == 1)
        if m.sum() >= min_per_group and (~m).sum() >= min_per_group:
            top_skill = s
            break
if top_skill is None:
    top_skill = top_skills[0]
print(f"        : {top_skill}")

col_name = "salary_usd"
tested_skill_mask = (df[top_skill] == 1)
NUM_REP = 10000
sample = df.sample(n=50000, random_state=42)
means_bootstrapped = bootstrap_mean(sample, tested_skill_mask, col_name,␣
 ↪NUM_REP)
# CI      95%
ci_lower, ci_upper = np.percentile(means_bootstrapped, [2.5, 97.5])
print(f"                95%: [{ci_lower:.2f}, {ci_upper:.2f}]")

mean_with    = df.loc[tested_skill_mask,  col_name].dropna().mean()
mean_without = df.loc[~tested_skill_mask, col_name].dropna().mean()
param_mean = mean_with - mean_without
```

```
        : Linux
                95%: [-445.12, 813.99]
```

[16]:
```
#
fig, ax = plt.subplots()
ax.hist(means_bootstrapped, bins=40, edgecolor='black', linewidth=0.6)
#        -
ax.hlines(y=0, xmin=ci_lower, xmax=ci_upper, colors='orange',␣
 ↪linestyles='solid', linewidth=6, zorder=5, clip_on=False)
#
ax.scatter([param_mean], [0], c='red', s=200, zorder=6, clip_on=False)
ax.set_title("Bootstrap Distribution of Difference in Salary Means")
ax.set_xlabel("Difference in Means (USD)")
ax.set_ylabel("Frequency")
fig.tight_layout()
plt.show()
```



4 Part 4

,

[17]:
```
# Build features: Education + Experience + Remote
skills_for_clustering = top24
```

```python
def standard_scale(df_in: pd.DataFrame):
    df = df_in.copy()
    means = df.mean(axis=0)
    stds = df.std(axis=0).replace(0, np.nan)  # avoid divide-by-zero
    return ((df - means) / stds).fillna(0.0)

# Compute per-skill averages for the three features
skill_avg_edu = {
    s: df.loc[df[s] == 1, "education_required_encoded"].mean()
    for s in skills_for_clustering if s in df.columns
}
skill_avg_exp = {
    s: df.loc[df[s] == 1, "experience_level_encoded"].mean()
    for s in skills_for_clustering if s in df.columns
}
skill_avg_remote = {
    s: df.loc[df[s] == 1, "remote_category_encoded"].mean()
    for s in skills_for_clustering if s in df.columns
}

# Create the feature table (three columns)
avg_df = pd.DataFrame({
    "AvgEducation": pd.Series(skill_avg_edu),
    "AvgExperience": pd.Series(skill_avg_exp),
    "AvgRemote": pd.Series(skill_avg_remote),
}).loc[skills_for_clustering]

# Scale to mean=0, std=1
km_scaled_df = standard_scale(avg_df)


# Numpy matrix for KMeans
X = km_scaled_df.values

print("Final features shape (should be N_skills x 3):", km_scaled_df.shape)
km_scaled_df.head()
```

```
Final features shape (should be N_skills x 3): (24, 3)
```

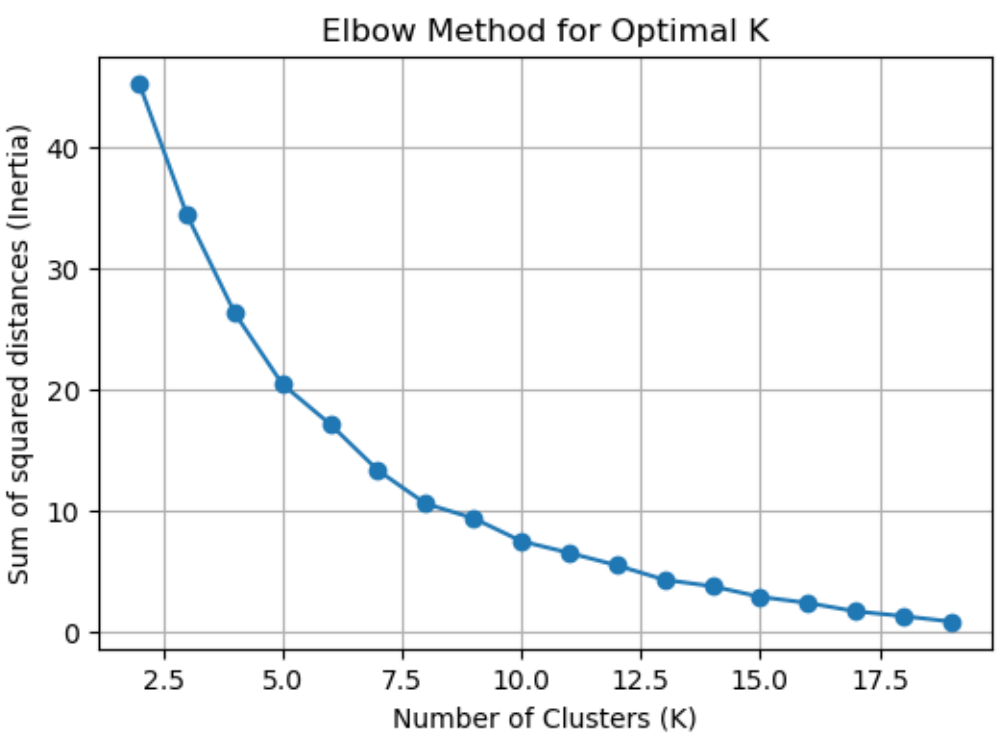
[17]:

	AvgEducation	AvgExperience	AvgRemote
Linux	0.021119	0.314033	0.893629
Docker	-0.771531	0.421227	0.537572
NLP	-0.463688	0.590546	-0.533845
Spark	-0.454021	1.384422	1.923649
GCP	1.031459	0.024117	-0.759029

, , .

K:

[18]:
```
# Elbow method
from sklearn.cluster import KMeans
import matplotlib.pyplot as plt

sse = []
K_range = range(2, 20)  # Try different K values

for k in K_range:
    kmeans = KMeans(n_clusters=k, random_state=42, n_init=10)
    kmeans.fit(km_scaled_df)
    sse.append(kmeans.inertia_)  # Inertia = sum of squared distances to␣
 ↪cluster centers

# Plot Elbow curve
plt.figure(figsize=(6, 4))
plt.plot(K_range, sse, marker='o')
plt.title("Elbow Method for Optimal K")
plt.xlabel("Number of Clusters (K)")
plt.ylabel("Sum of squared distances (Inertia)")
plt.grid(True)
plt.show()
```



[19]:
```python
# Run KMeans with chosen K
k_optimal = 6  # from elbow or decision

km = KMeans(n_clusters=k_optimal, random_state=42, n_init=10)
km.fit(km_scaled_df)

# Cluster centers (scaled space)
cluster_centers = km.cluster_centers_
labels = km.labels_

print("Cluster Centers Shape:", cluster_centers.shape)
print("Labels array:", labels)

# Assign clusters back to skills
df_clusters = km_scaled_df.copy()
df_clusters["Cluster"] = labels
df_clusters.index.name = "Skill"

# Optional: sort by cluster for readability
df_clusters_sorted = df_clusters.sort_values("Cluster")

df_clusters_sorted.head()
```

```
Cluster Centers Shape: (6, 3)
Labels array: [2 2 2 4 3 1 3 3 0 2 0 4 3 0 3 5 1 5 4 1 2 5 5 1]
```

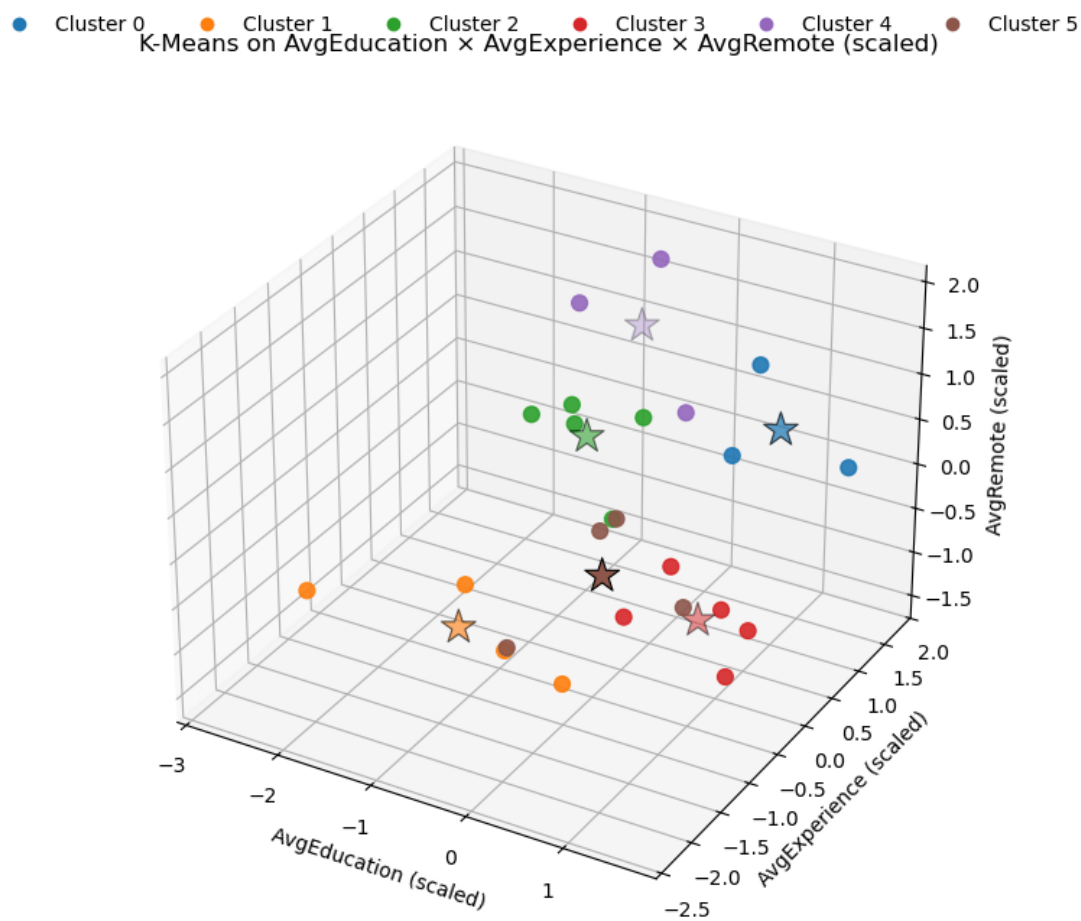
[19]:

Skill	AvgEducation	AvgExperience	AvgRemote	Cluster
TensorFlow	1.118693	0.480355	1.684788	0
Mathematics	1.619319	1.263546	0.284433	0
Scala	0.858948	0.465710	0.639882	0
Data Visualization	-0.959230	-1.114585	-0.422462	1
MLOps	0.091385	-1.158253	-1.167822	1

, :

[20]:
```python
# 3D visualization: Education × Experience × Remote

from mpl_toolkits.mplot3d import Axes3D  # needed for 3D projection

labels = km.labels_
k = km.n_clusters
cluster_colors = plt.cm.tab10(np.linspace(0, 1, max(10, k)))[:k]

fig = plt.figure(figsize=(7.5, 6.5))
ax = fig.add_subplot(111, projection='3d')
fig.suptitle("K-Means on AvgEducation × AvgExperience × AvgRemote (scaled)",␣
↪y=0.98)
```

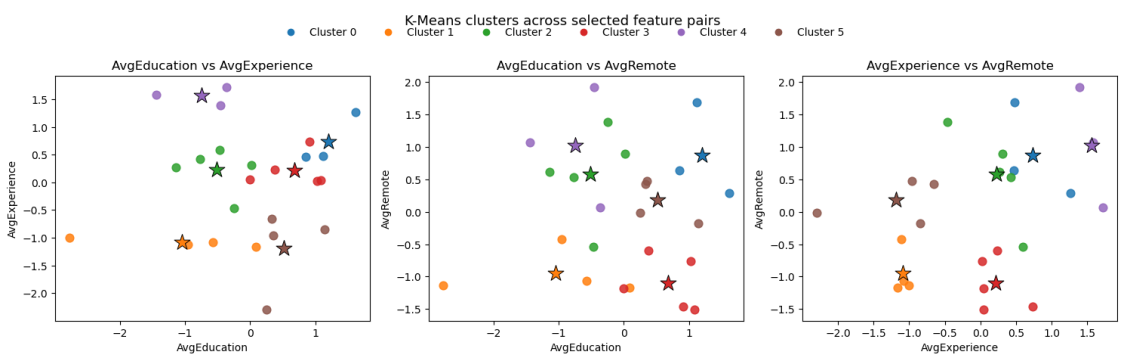
```python
# Points per cluster
for cl in range(k):
    m = (labels == cl)
    ax.scatter(
        km_scaled_df.loc[m, "AvgEducation"],
        km_scaled_df.loc[m, "AvgExperience"],
        km_scaled_df.loc[m, "AvgRemote"],
        s=60, color=cluster_colors[cl], alpha=0.9, label=f"Cluster {cl}"
    )

# Centroids
C = km.cluster_centers_
ax.scatter(
    C[:, 0], C[:, 1], C[:, 2],
    marker='*', s=320,
    c=[cluster_colors[cl] for cl in range(k)],
    edgecolor='black', linewidths=0.8
)

ax.set_xlabel("AvgEducation (scaled)")
ax.set_ylabel("AvgExperience (scaled)")
ax.set_zlabel("AvgRemote (scaled)")

# Legend above plot in one line
handles = [plt.Line2D([0],[0], marker='o', linestyle='',␣
 ↪color=cluster_colors[cl], label=f"Cluster {cl}") for cl in range(k)]
fig.legend(handles=handles, loc="upper center", ncol=min(k, 10), frameon=False,␣
 ↪bbox_to_anchor=(0.5, 1.02))

plt.tight_layout()
plt.show()
```



- XY, XZ, YZ

[21]:
```
# 2D plots for (Edu, Exp, Remote)
pairs = [
    ("AvgEducation", "AvgExperience"),
    ("AvgEducation", "AvgRemote"),
    ("AvgExperience", "AvgRemote"),
]

fig, axes = plt.subplots(1, 3, figsize=(15, 4.5))
for i, (fx, fy) in enumerate(pairs):
    ax = axes[i]
    for cl in range(k):
        m = (labels == cl)
        ax.scatter(
            km_scaled_df.loc[m, fx],
            km_scaled_df.loc[m, fy],
            s=60, color=cluster_colors[cl], alpha=0.85,
```

```
            label=f"Cluster {cl}" if i == 0 else None
        )
        # centroid for this pair (same color)
        ax.scatter(
            km.cluster_centers_[cl, km_scaled_df.columns.get_loc(fx)],
            km.cluster_centers_[cl, km_scaled_df.columns.get_loc(fy)],
            marker='*', s=250, color=cluster_colors[cl],
            edgecolor='black', linewidths=0.7, zorder=5
        )
    ax.set_xlabel(fx)
    ax.set_ylabel(fy)
    ax.set_title(f"{fx} vs {fy}")

# one shared legend above
handles = [plt.Line2D([0],[0], marker='o', linestyle='',␣
 ↪color=cluster_colors[cl], label=f"Cluster {cl}") for cl in range(k)]
fig.suptitle("K-Means clusters across selected feature pairs", y=1.03,␣
 ↪fontsize=13)
fig.legend(handles=handles, loc="upper center", ncol=min(k, 10), frameon=False,␣
 ↪bbox_to_anchor=(0.5, 1.02))

plt.tight_layout()
plt.show()
```



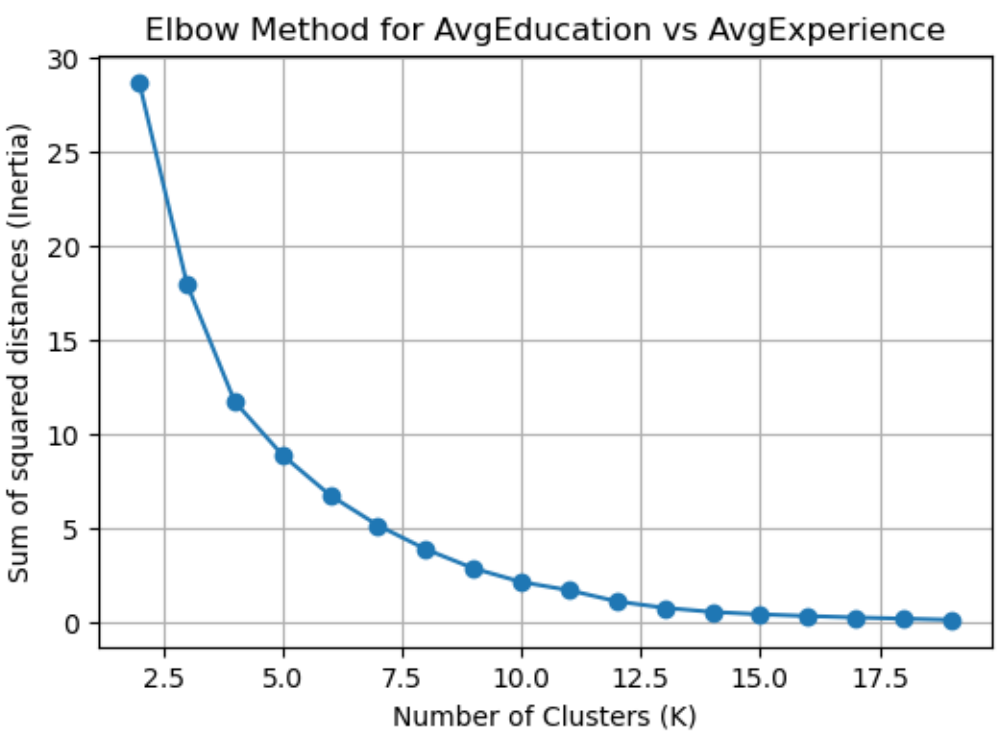
: ’

[22]:
```
# K-Means on PAIRS of features (Edu/Exp/Remote)

pairs = [
    ("AvgEducation", "AvgExperience"),
    ("AvgExperience", "AvgRemote"),
    ("AvgEducation", "AvgRemote"),
]
```

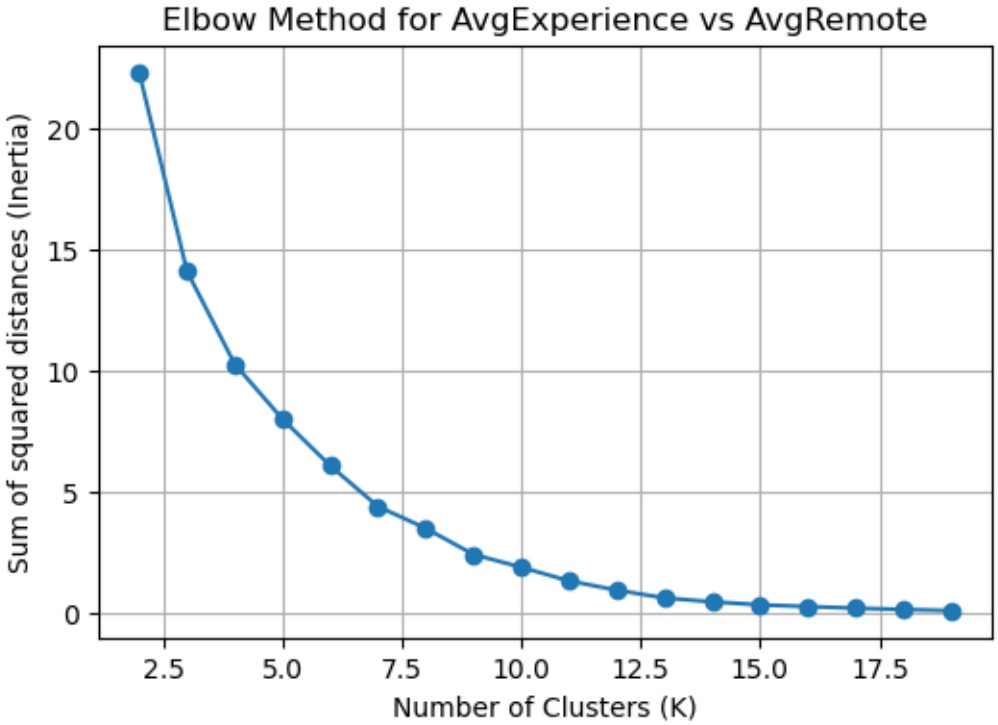
```python
def standard_scale(df):
    m, s = df.mean(0), df.std(0).replace(0, np.nan)
    return ((df - m) / s).fillna(0.0)

# Elbow plots for each pair
for fx, fy in pairs:
    print(f"\n Pair: {fx} vs {fy} ")

    df_pair_sc = standard_scale(avg_df[[fx, fy]])

    sse, K_range = [], range(2, 20)
    for k in K_range:
        km_tmp = KMeans(n_clusters=k, random_state=42, n_init=10)
        km_tmp.fit(df_pair_sc)
        sse.append(km_tmp.inertia_)

    plt.figure(figsize=(6, 4))
    plt.plot(K_range, sse, marker='o')
    plt.title(f"Elbow Method for {fx} vs {fy}")
    plt.xlabel("Number of Clusters (K)")
    plt.ylabel("Sum of squared distances (Inertia)")
    plt.grid(True)
    plt.show()

# Plot all pairs in one figure
k_optimal = 6  # Picked from elbow inspection
colors_fn = lambda k: plt.cm.tab10(np.linspace(0, 1, max(10, k)))[:k]

fig, axes = plt.subplots(1, 3, figsize=(16, 5))
axes = np.atleast_1d(axes)

for ax, (fx, fy) in zip(axes, pairs):
    df_pair_sc = standard_scale(avg_df[[fx, fy]])

    km = KMeans(n_clusters=k_optimal, random_state=42, n_init=10).
 ↪fit(df_pair_sc)
    labels, centers = km.labels_, km.cluster_centers_
    cols = colors_fn(k_optimal)

    for cl in range(k_optimal):
        mask = (labels == cl)
        ax.scatter(df_pair_sc.iloc[mask, 0], df_pair_sc.iloc[mask, 1],
                   s=60, alpha=0.9, color=cols[cl],
                   label=f"Cluster {cl}" if ax is axes[0] else None)
        ax.scatter(centers[cl, 0], centers[cl, 1],
                   marker='*', s=280, color=cols[cl],
                   edgecolor='black', linewidths=0.8, zorder=5)
```

```
        ax.text(centers[cl, 0], centers[cl, 1], f"  {cl}",
                fontsize=9, weight='bold', va='center')

    ax.set_xlabel(f"{fx} (scaled)")
    ax.set_ylabel(f"{fy} (scaled)")
    ax.set_title(f"{fx} vs {fy} (k={k_optimal})")
    ax.grid(True, alpha=0.25)
    for spine in ax.spines.values():
        spine.set_alpha(0.3)

# Legend & layout
handles, labels_legend = axes[0].get_legend_handles_labels()
fig.suptitle("K-Means clusters on pairs of features (scaled)", y=1.03,␣
 ↪fontsize=14)
fig.legend(handles, labels_legend, loc="upper center",
           ncol=min(len(handles), 10), frameon=False, bbox_to_anchor=(0.5, 1.
 ↪01))

plt.show()
```

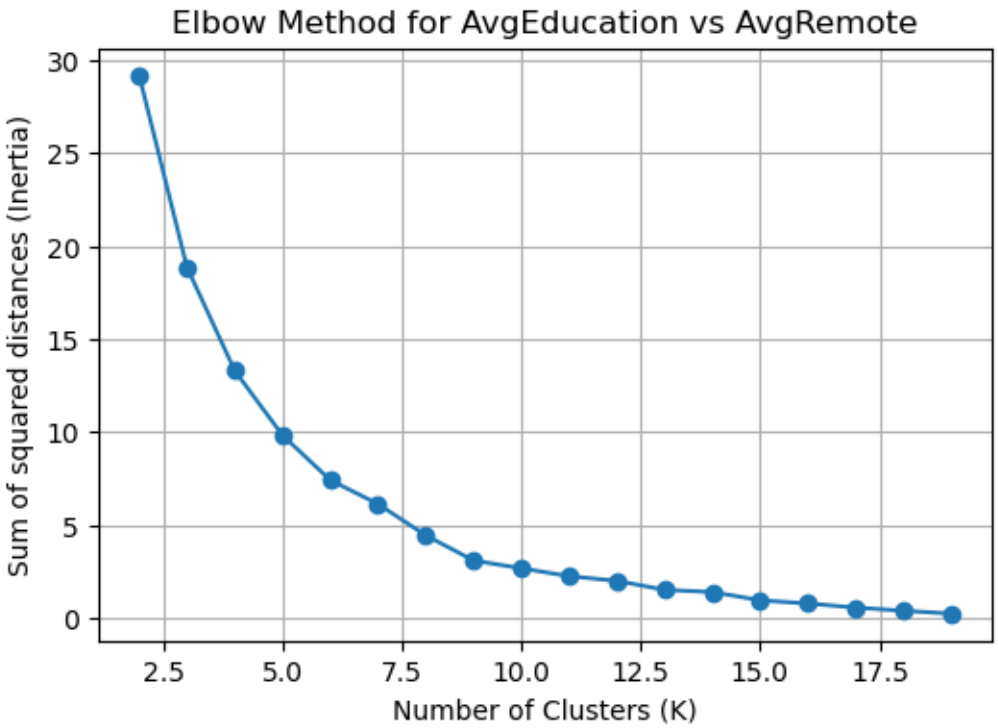
```
Pair: AvgEducation vs AvgExperience
```

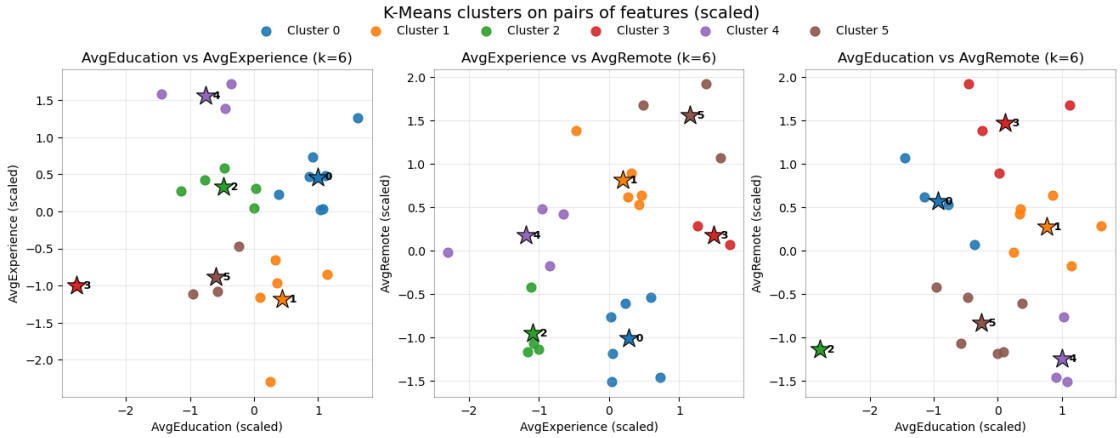


Pair: AvgExperience vs AvgRemote



Pair: AvgEducation vs AvgRemote


Elbow Method for AvgEducation vs AvgRemote
Sum of squared distances (Inertia)
Number of Clusters (K)


K-Means clusters on pairs of features (scaled)
Cluster 0
Cluster 1
Cluster 2
Cluster 3
Cluster 4
Cluster 5
AvgEducation vs AvgExperience (k=6)
AvgExperience vs AvgRemote (k=6)
AvgEducation vs AvgRemote (k=6)
AvgExperience (scaled)
AvgRemote (scaled)
AvgEducation (scaled)